From Source to Execution: Design and Implementation of a Statically-Typed Programming Language with Type Inference

by

Aye Khin Khin Hpone (Yolanda Lim), st125970

Applegate T. Tun Oo, st126690

Win Htut Naing, st126687

AT70.07 — Programming Languages and Compilers

Assignment 2

Instructor: Prof. Phan Minh Dung

Teaching Assistant: Akraradet Sinsamersuk

Asian Institute of Technology

School of Engineering and Technology

Thailand

April 2026

ABSTRACT

This report presents the design and implementation of a statically-typed interpreted programming language realised through a four-stage compiler pipeline. The language provides four primitive types — Integer, Float, Boolean, and String — and supports arithmetic and equality expressions, variable assignment with type inference from first use, conditional and iterative control flow, user-defined functions with value parameter passing, and a built-in callable `print()` function. Types are inferred from program context rather than declared explicitly, combining static type safety with a declaration-free syntax. The pipeline consists of a hand-written lexer, a recursive-descent parser that constructs an abstract syntax tree together with an inspectable parse-tree view, a static type checker that performs inference and rejects ill-typed programs before execution, and a tree-walking interpreter. The type checker implements a syntax-directed type inference algorithm over the abstract syntax tree, enforcing static type safety without explicit declarations and rejecting ill-typed programs before execution. Both a command-line runner and an interactive desktop interface expose each pipeline stage for inspection. The report documents the formal grammar, the type inference algorithm, the system architecture, and the automated test suite of 75 tests that validates the complete implementation.

CONTENTS

CHAPTER 1

INTRODUCTION

Programming languages define the formal contract between a programmer's intent and machine execution. Building a language processor from source text through to evaluated output requires a coherent pipeline of components, each of which must correctly transform its input before passing it downstream. Lexical analysis identifies atomic tokens; parsing imposes hierarchical structure; static type checking validates semantic correctness before execution; and interpretation evaluates the validated program. When these stages are connected cleanly, errors are caught at the earliest possible point, and the overall system remains tractable to reason about, test, and extend.

This report presents the design and implementation of a statically-typed interpreted language that realises this pipeline in full. The language supports four primitive types — Integer, Float, Boolean, and String — and provides arithmetic and equality expressions, assignment, conditional and iterative control flow, user-defined functions with value parameter passing, and a built-in callable `print()` function. Types are inferred from program context rather than declared explicitly, combining the safety of static typing with the conciseness of a declaration-free syntax. The implementation spans three principal components developed in pipeline order: the lexer and symbol table, the recursive-descent parser together with parse-tree and abstract-syntax-tree views, and the type checker with interpreter. Each component was completed before the next was begun, since each stage consumes the structured output of its predecessor.

The implemented system can be invoked from the command line or through an interactive desktop interface, both of which surface the five pipeline stages — tokens, parse tree, AST, inferred type table, and execution output — for inspection. The parse tree is provided as a grammar-oriented view for submission and debugging, while the AST remains the executable intermediate representation consumed by the type checker and interpreter. An automated regression suite of 75 tests verifies correctness across all stages and language features.

The main contributions of this project are:

- Design of a statically-typed programming language supporting Integer, Float, Boolean, and

String types without operator overloading.

- Implementation of a hand-written lexer and a recursive-descent parser with explicit parse-tree and abstract syntax tree generation.
- Development of a syntax-directed type inference and static type-checking system without explicit type declarations.
- Separation of integer and floating-point arithmetic operators following the Caml Light convention to eliminate semantic ambiguity.
- Construction of a complete end-to-end execution pipeline consisting of lexical analysis, parsing, semantic analysis, and interpretation.
- Development of both command-line and graphical interfaces exposing all compiler pipeline stages for inspection and debugging.
- Validation of the implementation through automated regression testing and semantic error verification.

The remainder of this report is organised as follows. Chapter 2 presents the language design, token specification, grammar, and operator precedence rules. Chapters 3 and 4 cover the front-end implementation in pipeline order: lexical analysis and the symbol table, then recursive-descent parsing and the AST. Chapter 5 explains the type inference algorithm and the semantic rules used by the checker. Chapter 6 presents the system architecture and implementation, including the end-to-end pipeline, project layout, and technology stack. Chapter 7 discusses testing and verification, and Chapter 8 concludes the report.

CHAPTER 2

LANGUAGE DESIGN AND GRAMMAR

2.1 Types

The language provides four primitive types, chosen to cover the core domains of numeric computation, logical control flow, and text manipulation while keeping the type system tractable and the semantics well-defined.

2.1.1 Integer

The Integer type represents whole numbers such as `0`, `7`, and `42`. Integer literals are written as one or more decimal digits without quotation marks or a decimal point.

2.1.2 Float

The Float type represents real-valued numbers written with a decimal point, such as `3.14` or `0.5`. Float values are used exclusively with the dot-suffixed arithmetic operators (`+.`, `-.`, `*.`, `/.`), following the Caml Light convention. Integer division (`/`) performs floor division and returns `Integer`: for example, `10 / 3` evaluates to `3 : Integer`.

The language intentionally does not perform implicit numeric promotion between `Integer` and `Float` values. This design decision follows directly from the assignment specification, which requires "no overloading with the usual precedence of multiplication and division over addition and subtraction." In a language where a single + operator accepts both $(\textit{Integer}, \textit{Integer}) \rightarrow \textit{Integer}$ and $(\textit{Integer}, \textit{Float}) \rightarrow \textit{Float}$, the operator effectively carries multiple type signatures — the compiler must inspect both operand types to determine the result type. This constitutes a form of operator overloading, even when described as "numeric promotion" or "widening coercion." To eliminate all such ambiguity, the language follows the Caml Light convention of using separate operator tokens for integer and floating-point arithmetic: `+`, `-`, `*`, `/` operate exclusively on `Integer` operands, while `+.`, `-.`, `*.`, `/.` operate exclusively on `Float` operands. Mixed-type arithmetic such as `1 + 2.5` is rejected by the type checker. Under this design, every operator token uniquely determines its operand types and result type — the compiler never needs to inspect operand types to resolve operator meaning, which is the defining property of a

language without operator overloading.

2.1.3 Boolean

The Boolean type represents logical truth values. The two Boolean literals are `true` and `false`. Boolean values are used in conditions and as results of comparison expressions.

2.1.4 String

The String type represents sequences of characters enclosed in double quotes, for example `"hello"`. Strings can be assigned to variables and printed through the built-in `print()` function. The surrounding quotes are stripped by the parser when the `Literal` node is created, so the value stored internally is the bare character sequence without surrounding quotes. At runtime, the built-in `print()` function re-adds double quotes around string values, so `print("plc")` outputs `"plc" : String`.

2.1.5 Void

Although the language exposes exactly four programmer-visible types, the implementation introduces a fifth value, `Void`, within the `LanguageType` enumeration. Its sole purpose is to provide a well-defined return type for functions that contain no `return` statement, allowing the symbol table to record a consistent entry for every function regardless of whether it produces a value. `Void` cannot appear in source code and is never visible to the programmer; it is a bookkeeping device used internally by the type checker. Therefore, `Void` does not extend the programmer-visible type system required by the assignment.

2.2 Static Typing

The language employs static typing, meaning that type errors are detected at type-checking time rather than at runtime. No explicit variable type declarations are required. Instead, the type checker infers types from literals, assignments, operations, function calls, and return statements. Once a variable type is established, later assignments must remain consistent with that type. For example, the following program is rejected at the type-checking stage before any execution occurs:

```
x = 5;
x = "hello";
```

The type checker rejects the second assignment with:

```
[line 2, col 1] TypeError: Cannot assign String to variable 'x' of type
    Integer
```

This contrasts with dynamically-typed languages, where the same program would run without error until (or unless) a type-dependent operation was attempted.

2.3 Token Specification

Table 2.1 summarises the major token classes used by the language.

Table 2.1. Token Specification

Token Category	Example Lexemes
`INT_LITERAL`	`0`, `10`, `42`
`FLOAT_LITERAL`	`3.14`, `20.5`
`BOOL_LITERAL`	`true`, `false`
`STRING_LITERAL`	`"hello"`
`IDENTIFIER`	variable and function names such as `x`, `add`
Arithmetic operators (integer)	`+`, `-`, `*`, `/`
Float arithmetic operators	`+.`, `-.`, `*.`, `/.`
Comparison operators	`==`, `!=`
Assignment operator	`=`
Keywords	`if`, `else`, `while`, `def`, `return`
Delimiters	`( ) { } , ;`

2.4 Context-Free Grammar

The language grammar can be described by the context-free grammar $G = (V_N, V_T, P, S)$, where V_N is the set of non-terminals, V_T is the set of terminals, P is the set of production rules, and S is the start symbol `program`. The grammar is implemented through recursive-descent parsing functions.

2.4.1 Program Structure

```
program   -> statement* EOF
statement -> assignment
           | if_stmt
           | while_stmt
           | func_def
           | return_stmt
           | expr ';'
```

2.4.2 *Expressions*

```
expr        -> comparison | additive
comparison  -> additive ('==' | '!=') additive
additive    -> term (('+' | '-' | '+.' | '-.') term)*
term        -> factor (('*' | '/' | '*.' | '/.') factor)*
factor      -> '-'  factor      (* integer unary negation *)
          | '-.' factor     (* float unary negation *)
          | INT_LITERAL
          | FLOAT_LITERAL
          | STRING_LITERAL
          | BOOL_LITERAL
          | IDENTIFIER
          | IDENTIFIER '(' args? ')'
          | '(' expr ')'
```

The `comparison` non-terminal is introduced so that the parse-tree renderer can label equality and inequality expressions explicitly. When an `expr` contains no `==` or `!=`, it expands directly to an `additive`; otherwise, it expands to a `comparison` node whose two operands are themselves `additive` sub-trees. Comparisons remain non-associative (one comparison per expression) and produce a `Boolean` result.

The operators `+`, `-`, `*`, and `/` operate exclusively on `Integer` operands. The dot-suffixed operators `+.`, `-.`, `*.`, and `/.` operate exclusively on `Float` operands. This follows the Caml Light convention of using distinct operator tokens for integer and float arithmetic, eliminating the need for implicit numeric promotion. Unary negation is similarly split: `-x` negates an `Integer` (desugared to `0 - x`), and `-.x` negates a `Float` (desugared to `0.0 -. x`).

Integer division and division by zero. `/` is floor (integer) division: both operands must be `Integer`, the result is `Integer`, and the value is $\lfloor a/b \rfloor$. For example, `7 / 2` evaluates to `3 : Integer`. Float division uses the distinct operator `/.` and requires two `Float` operands. Division by zero (`x / 0` or `x /. 0.0`) is detected at runtime and raises a `RuntimeError`.

2.4.3 *Boolean Expressions*

Comparison operators `==` and `!=` are the lowest-precedence binary operators and produce `Boolean` values. Because they are part of the general `expr` rule, Boolean expressions may appear both as control-flow conditions and as ordinary expressions. Their inferred type is `Boolean`, allowing comparison results to be assigned to variables, passed as arguments, or printed:

```
flag = (x == y);
print(a != b);
```

Here `flag` is inferred as `Boolean` from the assignment, and `print(a != b)` outputs `true : Boolean` or `false : Boolean`.

Strict arithmetic operands. The specification states that == and != operate between two arithmetic expressions. We interpret "arithmetic" strictly as `Integer` and `Float`, consistent with the no-overloading rule. Both operands must also share the same numeric type (no implicit coercion). Therefore `Integer == Integer` and `Float == Float` are well-typed; `Integer == Float`, `Boolean == Boolean`, and `String == String` are all rejected at type-check time.

2.4.4 Statements

```
assignment  -> IDENTIFIER '=' expr ';'
if_stmt     -> 'if' '(' expr ')' block ('else' block)?
while_stmt  -> 'while' '(' expr ')' block
return_stmt -> 'return' expr ';'
block       -> '{' statement* '}'
```

Design note: `print` as a built-in callable

`print` is implemented as a normal function-call expression rather than as a dedicated statement form. Lexically, `print` is tokenised as an `IDENTIFIER`; syntactically, `print(x)` is parsed through the same `FunctionCall` path used for user-defined calls. Semantically it is a unary built-in of type `T` $\rightarrow$ `()`: it accepts one argument of any programmer-visible type, emits `value : Type` immediately at runtime, and returns the internal `Void` type. As with any void-returning call, `print(...)` may be used as a standalone expression statement but not as the right-hand side of an assignment.

2.4.5 Function Definitions and Calls

```
func_def -> 'def' IDENTIFIER '(' params? ')' block
params   -> IDENTIFIER (',' IDENTIFIER)*  | epsilon
args     -> expr (',' expr)*              | epsilon
```

The `?` suffix on `params?` in `func_def` (and on `args?` in the call form below) marks the construct as optional in EBNF form. The corresponding parse-tree renderer makes this explicit by emitting a single `epsilon` leaf as the only child of an empty `params` or `args` node, so an empty parameter or argument list is still labelled in the tree.

Function calls use value parameter passing. This means the argument values are evaluated before the call and copied into the function's local environment.

The built-in `print()` function uses exactly the same surface syntax as other calls:

```
print(x);
print(3.14);
```

Return type uniformity. A function's return type is inferred from its `return` statements. All `return` expressions within a function body must yield the same type; mixed return types are a compile-time error. A function with no `return` statement is assigned the special internal type `Void` and may only be invoked as a standalone statement, not used as an expression. Assigning the result of a `Void` function to a variable (e.g. `x = f();` where `f` returns nothing) is rejected by the type checker.

2.5 Operator Precedence and Associativity

Operator precedence is encoded structurally in the recursive-descent parser. Comparison is lowest, addition and subtraction are next, and multiplication and division have the highest precedence among binary operators. Arithmetic operators are left-associative.

Table 2.2. Operator Precedence (1 = lowest, 3 = highest) and Associativity

Precedence	Operators	Associativity
1 (lowest)	`==`, `!=`	non-associative (one comparison per expression)
2	`+`, `-`, `+.`, `-.`	left-associative
3 (highest)	`*`, `/`, `*.`, `/.`	left-associative

Table 2.3. Formal arithmetic and comparison operator signatures

Operator	Operand types	Result type
`+`	Integer × Integer	Integer
`-`	Integer × Integer	Integer
`*`	Integer × Integer	Integer
`/`	Integer × Integer	Integer
`+.`	Float × Float	Float
`-.`	Float × Float	Float
`*.`	Float × Float	Float
`/.`	Float × Float	Float
`==`	Integer × Integer	Boolean
`==`	Float × Float	Boolean
`!=`	Integer × Integer	Boolean
`!=`	Float × Float	Boolean

Any operand combination not listed in Table 2.3 is rejected during static type checking. Because each operator token maps to exactly one valid operand signature (or, for `==` and `!=`,

two same-type signatures), operator meaning can be determined without overload resolution or implicit type coercion.

2.6 Design Decisions: No Overloading

The assignment specification states: "No overloading with the usual precedence of multiplication and division over addition and subtraction. Hence no explicit declaration." This section explains how the language design satisfies this requirement and why alternatives were rejected.

Definition of overloading. An operator is overloaded when the same syntactic symbol carries more than one type signature, requiring the compiler to inspect operand types to determine the operator's meaning or result type. Under this definition, a language where + means both integer addition $(\mathit{Integer}, \mathit{Integer}) \to \mathit{Integer}$ and floating-point addition $(\mathit{Float}, \mathit{Float}) \to \mathit{Float}$ — or worse, mixed-type addition $(\mathit{Integer}, \mathit{Float}) \to \mathit{Float}$ — overloads the + symbol.

Our approach: distinct operator tokens. Following the Caml Light convention, this language assigns a unique token to each arithmetic meaning. Integer arithmetic uses `+`, `-`, `*`, `/`; floating-point arithmetic uses `+.`, `-.`, `*.`, `/.`. Each token has exactly one type signature. The type of any expression can be determined from the operator token alone, without inspecting operand types. This is the strongest possible interpretation of "no overloading."

Rejected alternative: implicit numeric promotion. An alternative design would use a single set of operators and silently promote `Integer` operands to `Float` when mixed with a `Float` operand (e.g. `1 + 2.5` $\to$ `3.5 : Float`). Proponents of this approach argue that promotion is not overloading because the operator retains a single "arithmetic addition" meaning. However, this argument is difficult to sustain formally: the operator + would have three distinct input-type combinations — $(\mathit{Integer}, \mathit{Integer})$, $(\mathit{Integer}, \mathit{Float})$, and $(\mathit{Float}, \mathit{Float})$ — with two different result types, a dispatch table that the compiler must resolve from operand types, which is the hallmark of overloading. The assignment specification also states "hence no explicit declaration," implying that type inference should be unambiguous; implicit promotion introduces cases where the inferred result type depends on operand types rather than on the operator itself.

Connection to type inference. Because every operator token uniquely determines its operand and result types, the type inference algorithm described in Chapter 5 can determine the type of any expression without backtracking, unification, or constraint solving. This determinism directly satisfies the specification's coupling of "no overloading" with "no explicit declaration."

Table 2.4 contrasts the two designs.

Table 2.4. Comparison of operator design approaches

Expression	**This language (separate tokens)**	**Promotion-based language**
`3 + 4`	`7 : Integer` (+ → Integer)	`7 : Integer`
`3.0 +. 4.0`	`7.0 : Float` (+. → Float)	N/A (no `+.` operator)
`3 + 4.0`	`TypeError`: + requires two Integer operands	`7.0 : Float` (implicit promotion)
`10 / 3`	`3 : Integer` (floor division)	`3.333... : Float` (or `3 : Integer`, design-dependent)
Type signatures of +	One: $(Int, Int) \to Int$	Three: (Int, Int), $(Int, Float)$, $(Float, Float)$
Overloaded?	No	Arguably yes

CHAPTER 3

LEXICAL ANALYSIS AND SYMBOL TABLE

This chapter covers the responsibilities related to lexical and early semantic foundations of the language. It includes defining token structures and categories (such as keywords, operators, identifiers, and literals), implementing the lexer to transform source characters into tokens, and establishing the symbol table to manage variable and function storage along with their associated types. Together, these components ensure that raw source input is systematically organised into meaningful structures that can be used reliably by later stages of the compiler.

3.1 Token representation

The token system, written in `components/tokens.py`, establishes the vocabulary and structure of all lexical elements in the language. It specifies both the types of tokens that can exist and the structure each token must follow. This serves as a canonical contract between the lexer and the parser, ensuring consistency across all stages of the compiler.

Table 3.1 summarises the main categories, example lexemes, and typical `TokenType` values.

Table 3.1. Token categories and representative `TokenType` values

Category	Example lexemes	Representative `TokenType`
Literals	`42`, `3.14`, `true`, `"hello"`	`INT_LITERAL`, `FLOAT_LITERAL`, `BOOL_LITERAL`, `STRING_LITERAL`
Identifiers	`x`, `counter`, `add`	`IDENTIFIER`
Keywords	`if`, `else`, `while`, `def`, `return`	keyword-specific token variants
Integer operators	`+`, `-`, `*`, `/`	`PLUS`, `MINUS`, `STAR`, `SLASH`
Float operators	`+.`, `-.`, `*.`, `/.`	`PLUS_DOT`, `MINUS_DOT`, `STAR_DOT`, `SLASH_DOT`
Other operators	`=`, `==`, `!=`	`ASSIGN`, `EQUAL_EQUAL`, `BANG_EQUAL`
Delimiters	Parentheses, braces, comma, and semicolon (see lexer delimiter tokens)	delimiter token variants
End marker	end of source input	`EOF`

Token types (from class `TokenType`) are defined using an `Enum` with `auto()` to automatically assign unique values. This avoids reliance on raw strings and ensures safer comparisons during parsing. The token categories include literals, identifiers, keywords, operators, delimiters,

and a special end-of-file (EOF) token. The EOF token is appended after all input is processed, allowing the parser to detect the end of input explicitly rather than encountering an unexpected termination.

Each token is represented as an immutable dataclass (frozen=True), meaning its fields cannot be modified after creation. Each token contains four key pieces of information: its kind (token_type), its lexeme (the exact substring from the source code), and its source position (line, column). This structured representation allows later stages to process the program in terms of syntax and errors without handling raw text directly. Immutability improves reliability by preventing accidental modification during later processing.

```
@dataclass(frozen=True)
class Token:
    token_type: TokenType
    lexeme: str
    line: int
    column: int
```

3.2 Lexer implementation

The lexer, implemented in components/lexica.py, is a hand-written scanner that converts raw source text into a sequence of tokens. Unlike generated lexers (for example SLY lexer mode, or Lex and Flex), this implementation uses explicit control flow (if and while) to process input.

The lexer maintains internal state including the source string, current position, start of the current token, line number, and column index. The column index is tracked per line and resets upon encountering a newline, while a separate token_column records the starting position of each token for accurate error reporting.

The main function, tokenize, iterates through the source code, repeatedly invoking _scan_token to identify the next token. Tokens are appended to a list, and an EOF token is added at the end. Whitespace is skipped, while invalid input results in a LexerError.

```
def tokenize(self) -> list[Token]:
    tokens: list[Token] = []
    while not self._is_at_end():
        self.start = self.current
        self.token_column = self.column
        token = self._scan_token()
        if token is not None:
            tokens.append(token)
    tokens.append(Token(TokenType.EOF, "", self.line, self.column))
```

```
    return tokens
```

Token recognition follows a structured approach. Single-character tokens are mapped using a lookup table. Multi-character operators such as `==` and `!=` are handled using lookahead via `_match`. String literals are processed by `_string`, which reads characters until a closing quotation mark or the end of input is reached, with error handling for unterminated strings. Numeric literals are handled by `_number`, which distinguishes integers and floating-point values based on the presence of a decimal point followed by digits. Identifiers are processed greedily, consuming alphanumeric characters and underscores, and then checked against the `KEYWORDS` map to determine whether they are reserved words or user-defined names. Notably, `true` and `false` are also entries in this map, mapping to `TokenType.BOOL_LITERAL` rather than a keyword-specific type. This means boolean literals are recognised through the same keyword-fallback mechanism as reserved words, preventing them from being scanned as user-defined identifiers. In contrast, `print` is *not* reserved: it is scanned as an ordinary `IDENTIFIER` and later treated as a built-in callable by the type checker and interpreter.

```
KEYWORDS: dict[str, TokenType] = {
    "if": TokenType.IF,
    "else": TokenType.ELSE,
    "while": TokenType.WHILE,
    # ...
    "true": TokenType.BOOL_LITERAL,
    "false": TokenType.BOOL_LITERAL,
}
```

```
# Arithmetic operators with optional trailing '.' for float variant
_ARITH_DOT: dict[str, tuple[TokenType, TokenType]] = {
    "+": (TokenType.PLUS,  TokenType.PLUS_DOT),
    "-": (TokenType.MINUS, TokenType.MINUS_DOT),
    "*": (TokenType.STAR,  TokenType.STAR_DOT),
    "/": (TokenType.SLASH, TokenType.SLASH_DOT),
}
```

When one of these characters is encountered, the lexer checks the next character: if it is a `.` (and the character after that is not a digit), the dot-suffixed float variant is emitted; otherwise the plain integer operator is emitted. This lookahead prevents the `.` in a float literal such as `3.14` from being consumed as part of an operator.

```
SINGLE_CHAR_TOKENS: dict[str, TokenType] = {
    "=": TokenType.ASSIGN,
    "(": TokenType.LPAREN,
    ")": TokenType.RPAREN,
    # ...
```

```
}
```

Helper functions such as `_advance`, `_peek`, and `_peek_next` manage character consumption and lookahead, while `_is_at_end` determines when input has been fully processed. Error messages include precise line and column information, aiding debugging and diagnostics.

Table 3.2 lists the main functions and data structures in `components/lexica.py` and their roles.

Table 3.2. Key functions and data in `lexica.py`

Function / data	Responsibility	Output / effect
`tokenize()`	Iterates through source, scans tokens, appends `EOF`	ordered token list
`_scan_token()`	Dispatches recognition based on character and lookahead	next token or skip
`_number()`	Parses numeric sequences, distinguishes int vs float	numeric literal token
`_string()`	Parses quoted text, checks termination	string token or error
`_identifier()`	Parses identifiers, resolves keyword vs user-defined	keyword or identifier token
`KEYWORDS` map	Maps reserved words and Boolean literals to token types	token type selection
`SINGLE_CHAR_TOKENS` map	Maps single-character symbols	token type selection
`_advance()`, `_peek()`, `_match()`	Character consumption and lookahead	scanner state update

3.3 Symbol table and semantic structure

The symbol table, defined in `components/symbol_table.py`, operates at a higher level than the lexer. While the lexer processes raw text into tokens, the symbol table manages semantic information such as variable and function names, their types, and their scope.

The symbol table is implemented as a scoped structure, where each scope maintains its own dictionary of symbols and optionally references a parent scope. This enables proper handling of nested blocks and variable shadowing. Symbols are introduced through definition functions and retrieved through lookup operations, which search recursively through parent scopes if needed.

```
def lookup(self, name: str) -> Symbol:
    symbol = self._symbols.get(name)
    if symbol is not None:
        return symbol
    if self.parent is not None:
```

```
6             return self.parent.lookup(name)
7         raise SymbolTableError(f"Undefined symbol: {name}")
```

The type system is represented using a `LanguageType` enumeration, which includes fundamental types such as Integer, Float, Boolean, String, and Void. Symbols are modelled using an inheritance hierarchy: the base `Symbol` class stores a name and type, while subclasses such as `VariableSymbol` and `FunctionSymbol` include additional attributes. Variables track initialization state, while functions store parameter lists and return types. Table 3.3 summarises the main building blocks.

Table 3.3. Symbol table components and roles

Component	Stored information	Role in semantic analysis
Scoped symbol table	Symbols in current and parent scope	Supports nesting and shadowing via recursive lookup
`LanguageType` enum	Integer, Float, Boolean, String, Void	Defines the type system
`Symbol` (base)	Name and declared or inferred type	Base abstraction for entries
`VariableSymbol`	Variable metadata (including initialization state)	Tracks correct usage before assignment
`FunctionSymbol`	Parameters and return type	Validates calls and return consistency
`lookup(name)`	Recursive scope search	Resolves identifiers or reports undefined
`SymbolTableError`	Semantic error details	Reports duplicates, undefined symbols, invalid operations

Errors related to semantic violations are handled via `SymbolTableError`. These include duplicate declarations within the same scope, references to undefined symbols, and invalid operations such as attempting to mark a non-variable symbol as initialized.

3.3.1 Example symbol table hierarchy

Figure 3.1 illustrates a small two-scope symbol table after the type checker has processed a program containing two top-level variables and a function definition.

```
 Global scope
──────────────────────────────────────
Name   Kind       Type
──────────────────────────────────────
x      Variable   Integer
y      Variable   Float
addf   Function   Float × Float → Float

 Function scope:  addf
──────────────────────────────
Name   Kind        Type
──────────────────────────────
a      Parameter   Float
b      Parameter   Float
```

Figure 3.1. Example symbol table hierarchy with one global scope and one function scope.

The global scope stores top-level variables and function definitions, while each function invocation creates a child scope containing parameter bindings and local variables. Lookup operations recursively search parent scopes when identifiers are not found in the current environment.

3.4 Role of the symbol table in the pipeline

The symbol table is constructed during the type-checking phase rather than during lexical analysis. This guideline is followed in the implementation. The type checker initialises a global symbol table and populates it while traversing the AST, including defining variables and functions, managing nested scopes, and validating type correctness.

```
def run_pipeline(source_code: str) -> PipelineResult:
    tokens = Lexer(source_code).tokenize()
    tree = Parser(tokens).parse()
    parse_tree_text = ParseTreePrinter().print(tree)
    ast_text = ASTPrinter().print(tree)
    checked_scope = TypeChecker().check(tree)
    outputs: list[str] = []
    Interpreter(output_callback=outputs.append).execute(tree)
    return PipelineResult(
        tokens=tokens,
        parse_tree_text=parse_tree_text,
        ast_text=ast_text,
        checked_scope=checked_scope,
        outputs=outputs,
    )
```

Once type checking is complete, the populated symbol table is returned as part of the pipeline result (`checked_scope`). The checker creates child scopes for blocks and functions during analysis, but the displayed `checked_scope` table represents the global scope entries returned

by the pipeline. It can then be displayed or used for further analysis.

This separation of concerns keeps lexical analysis focused on tokenization, while semantic meaning is enforced during type checking. For user-defined functions, definitions are first registered in the global scope with placeholder parameter and return types. These are inferred from the first valid call and refined through body checking; subsequent calls must match the inferred parameter types.

CHAPTER 4

RECURSIVE-DESCENT PARSING, PARSE TREE, AND ABSTRACT SYNTAX TREE

This chapter presents the implementation of the recursive-descent parser, the grammar-oriented parse-tree view exported for inspection, and the associated abstract syntax tree (AST) model used for downstream semantic analysis and execution. Starting from lexer-produced tokens, the parser constructs a hierarchical `Program` AST. A separate `ParseTreePrinter` then renders that parsed program in an ASCII branch-style rule-oriented tree form so that the submission can show explicit correspondence between grammar productions and accepted programs without changing the executable internal representation. Implemented syntax coverage includes arithmetic expressions, assignments, if-then-else, while, function definitions, function calls, return, and built-in `print(...)` calls. Equality operators (`==`, `!=`) are parsed as ordinary lowest-precedence expressions, so Boolean results can appear in conditions, assignments, function arguments, and `print(...)` expressions.

4.1 Abstract syntax tree representation

The AST is defined as a collection of Python dataclasses representing the structural elements of the programming language. It serves as an intermediate representation between parsing and later stages such as type checking and interpretation.

A key distinction exists between tokens and AST nodes. Tokens are flat and describe individual lexical elements such as operators or identifiers. In contrast, the AST is hierarchical. For example, while a token may represent the symbol `+`, a `BinaryOp` node represents an entire addition operation, including its left and right operands as subtrees.

All AST nodes inherit from a common base class, allowing them to store positional metadata (`line`, `column`). This ensures that errors detected in later stages can still reference precise locations in the original source code.

```
1 @dataclass
2 class ASTNode:
3     """Base class for every AST node. Carries source location for
          diagnostics."""
4     line: int = field(default=0, kw_only=True)
```

```
    column: int = field(default=0, kw_only=True)
```

4.1.1 *Expressions*

Expressions represent computations or values within the program. A `Literal` node stores values known at parse time, including integers, floating-point numbers, booleans, and strings. String literals are stored without surrounding quotation marks, as these are removed during parsing.

```
if tok.token_type == TokenType.STRING_LITERAL:
    self._advance()
    return Literal(value=tok.lexeme[1:-1], line=tok.line, column=tok.
        column)
```

An `Identifier` node represents the use of a variable name within an expression, indicating evaluation rather than declaration. A `BinaryOp` node models arithmetic and comparison operations such as addition, subtraction, multiplication, division, and equality checks, storing the operator along with references to left and right operands. A `FunctionCall` node represents function invocation, consisting of the function name and a list of argument expressions.

```
@dataclass
class BinaryOp(ASTNode):
    """An arithmetic or comparison binary operation.

    op is one of: '+' '-' '*' '/' '+.' '-.' '*.' '/.' '==' '!='
    Example:  a + b    x == 0    a +. b
    """
    op: str
    left: ASTNode
    right: ASTNode
```

4.1.2 *Statements*

Statements represent actions or control flow within the program. An `Assign` node models variable assignment, pairing a variable name with a value expression, while `Return` stores a single expression and represents function return. A `FunctionCall` node may also appear as a standalone statement, which is how built-in `print(...)` calls are represented after parsing. A `Block` node groups multiple statements within braces, representing a scoped execution sequence.

Control flow is handled via `If` and `While` nodes. The `If` node stores a condition, a then-block, and an optional else-block, while `While` stores a loop condition and body block. Function

definitions are represented by `FunctionDef` nodes, which include the function name, a list of parameter names, and a body block. Parameter types are intentionally not assigned at parse time, as type inference is handled later by the type checker. Finally, `Program` serves as the AST root, containing all top-level statements.

4.2 Parser implementation

The parser consumes the list of tokens generated by the lexer and produces a single `Program` AST node. It is implemented as a hand-written recursive-descent parser based on the project grammar. This avoids parser generators and instead relies on explicit functions for each grammar rule.

The parser maintains internal state consisting of the token list and a position index (`self._pos`) indicating the current token. Tokens are accessed via helper methods such as `_peek` and `_advance`. Parsing errors are reported using a custom `ParseError` exception with line and column information consistent with lexer diagnostics.

```
def parse(self) -> Program:
    """Parse the full token stream and return the root Program node."""
    line, col = self._peek().line, self._peek().column
    statements: list[ASTNode] = []
    while not self._is_at_end():
        statements.append(self._statement())
    return Program(statements=statements, line=line, column=col)
```

In addition to the main `parse()` entry point, the parser's statement dispatcher (`_statement`) implements a deterministic decision order over token patterns: function definition, conditional, loop, return, assignment, then expression statement fallback. This design is important because it prevents ambiguity between syntactically similar forms. In particular, assignments are recognized using one-token lookahead (`IDENTIFIER` followed by `=`), while identifier-led expressions that are not assignments are parsed as expression statements and must still end with a semicolon. This gives the parser full coverage of both declarative-style and expression-driven top-level statements without requiring a parser generator.

4.2.1 Rationale for recursive-descent parsing

Recursive-descent parsing was selected because the project grammar is sufficiently structured to permit predictive parsing with explicit precedence layering. This approach avoids parser generators and keeps parsing logic directly aligned with grammar productions, making the

implementation easier to debug, extend, and explain.

Operator precedence is encoded structurally through layered parsing methods (`_expr`, `_additive`, `_term`, `_factor`), eliminating the need for runtime precedence tables. The hand-written approach also provides explicit control over lookahead and ambiguity resolution, particularly for distinguishing assignments from identifier-led expression statements.

4.3 Parse-tree view

Although the parser's internal product is an AST, the project now also exposes a separate parse-tree view for reporting and demonstration. This view is generated by `components/parse_tree_printer.py` from the parsed program and uses grammar-oriented layers such as `program`, `statement`, `assignment`, `expr`, `comparison`, `additive`, `term`, `factor`, `function_call`, and `args`. The `comparison` layer wraps the operands of `==` and `!=` when they appear at the top of an expression. Terminal-like leaves are rendered as `IDENTIFIER(x)`, `INT_LITERAL(5)`, `'('`, `';'`, and `EOF`, with ASCII branch markers indicating parent-child structure.

This design keeps the semantic implementation simple: the type checker and interpreter still consume the AST, while the parse-tree renderer exists purely to expose the grammar structure required for inspection.

Table 4.1. Parse tree versus AST in the implemented system

Artifact	Purpose	Used by
Parse tree	Grammar-oriented inspection view	CLI, UI, report
AST	Executable semantic tree	Type checker, interpreter

For presentation purposes, Figure 4.1 gives a tiny textbook-style diagram for the source fragment `a = 3;`. This figure is included only as an explanatory visual in the report; the actual system output uses the more scalable ASCII branch representation shown in the CLI and UI.

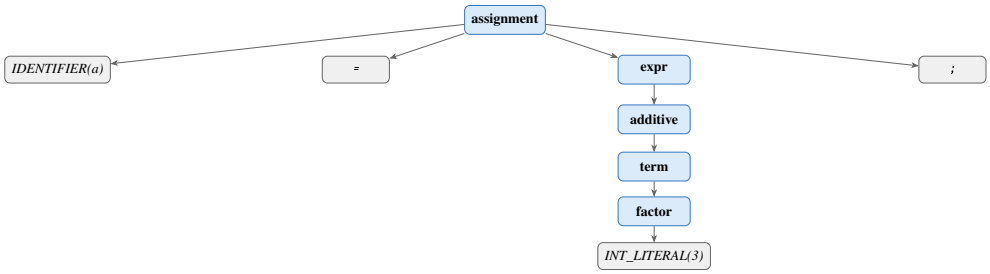


Figure 4.1. Tiny textbook-style parse-tree diagram for `a = 3;`.

4.3.1 *Statement parsing*

The main parsing loop repeatedly processes statements until `EOF`, with the statement type determined via dispatch on the current token. Function definitions, conditionals, loops, and returns each have dedicated parsing methods. Assignments are detected using one-token lookahead (`IDENTIFIER` followed by `ASSIGN`), preventing misclassification of function calls as assignments. If no specific statement pattern matches, the parser falls back to an expression statement and enforces a terminating semicolon.

```
# assignment: IDENTIFIER "=" expr ";"
if (
    tok.token_type == TokenType.IDENTIFIER
    and self._peek_next().token_type == TokenType.ASSIGN
):
    return self._assignment()
# fallback: bare expression statement  expr ";"
node = self._expr()
self._expect(TokenType.SEMICOLON, "Expected ';' after expression")
```

4.3.2 *Expression parsing and precedence*

Expressions are parsed in layered precedence levels: `_expr` handles comparison operators first (`==`, `!=`) after parsing an additive left-hand side, then `_additive` handles `+`, `-`, `+.`, and `-.`; `_term` handles `*`, `/`, `*.`, and `/.`; and `_factor` handles literals, identifiers and function calls, and parenthesized expressions. Function calls are recognised when an identifier is followed by `(`, with argument lists parsed as comma-separated expressions. Parenthesized expressions are supported to override precedence. Unary negation (`-x`, `-5`) is also handled at the `_factor` level: a leading `-` token causes the parser to recursively parse the operand and return `BinaryOp(0 - operand)`, so the existing type rules for subtraction cover negation without any additional inference code. Float unary negation is handled similarly for `-.`, producing `BinaryOp(0.0 -. operand)`.

```
def _expr(self) -> ASTNode:       # ==, !=
    ...
def _additive(self) -> ASTNode:   # +, -, +., -.
    ...
def _term(self) -> ASTNode:   # *, /, *., /.
    ...
def _factor(self) -> ASTNode:   # unary -, literals, identifiers/calls,
    grouped expr
    ...
```

Figure 4.2 shows the AST produced for the statement `z = x + y * 2;`. Because `_term` han-

dles * before _additive handles +, the multiplication sub-tree is nested one level deeper, encoding the higher precedence of * over + without any explicit priority table at runtime.

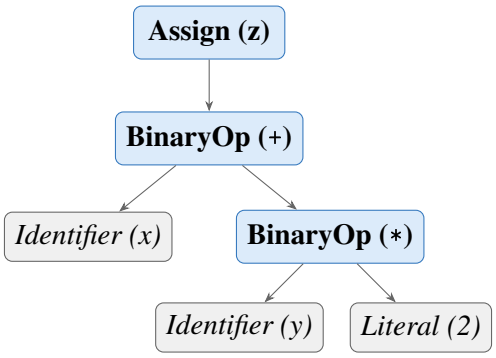


Figure 4.2. AST for `z = x + y * 2;`. Multiplication is a deeper sub-tree than addition, encoding higher precedence through tree structure rather than a runtime priority table.

This structural encoding of precedence eliminates the need for runtime operator-priority resolution and keeps semantic analysis independent of parsing strategy.

Expression parsing is intentionally split into layered methods to enforce precedence and associativity in a transparent way. Parenthesized expressions are parsed explicitly and re-enter `_expr`, allowing precedence override where required by source syntax. Comparison operators (==, !=) are parsed at the top of the `_expr` method as the lowest-precedence binary operators. Because comparisons are part of the general expression grammar, Boolean results can appear in any expression context — as conditions, assignments, arguments, or print expressions. Semantic enforcement that a condition is truly Boolean is deferred to the type checker.

Figure 4.3 shows the AST for an `if` statement with a comparison condition, illustrating how the parser structures control flow with a Boolean sub-tree.

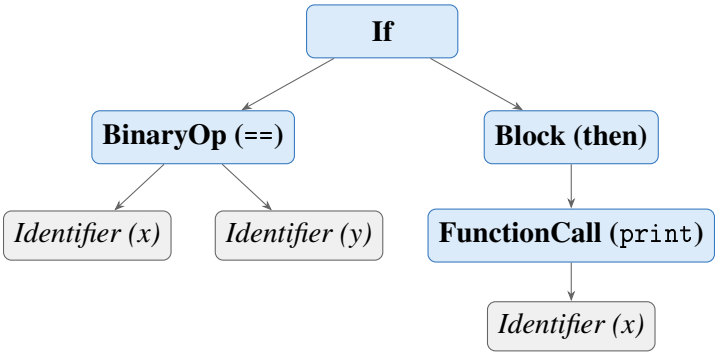


Figure 4.3. AST for `if (x == y) { print(x); }`. The comparison produces a `BinaryOp` node whose inferred type is `Boolean`, checked by the type checker before execution.

4.3.3 Helper functions and lookahead

As described above, the parser uses `_check`, `_expect`, `_peek`, `_peek_next`, `_advance`, and `_is_at_end` for token navigation and validation. Lookahead is central for ambiguity resolution: assignment versus function call is resolved by inspecting the next token, while comparison operators are recognised in `_expr` after parsing the left additive expression. Parser errors follow a consistent diagnostic format: `[line X, col Y] ParseError: <message>`.

The parser also includes dedicated list parsers for formal parameters (`_params`) and call arguments (`_args`), both implemented as comma-separated sequences with optional emptiness. This ensures consistent handling of signatures such as `def f()` and `def f(a, b)` as well as invocations like `f()` and `f(x, y+1)`.

4.4 AST printing

For debugging and reporting, an AST printer converts the tree into a readable indented string, integrated into the pipeline output. The printer uses visitor-style dynamic dispatch, mapping node class names to methods such as `_visit_If` and `_visit_BinaryOp`, with output built as multi-line text where indentation reflects tree depth. Node formatting emphasises structure: `Program` and `Block` include statement counts, `BinaryOp` explicitly shows left and right subtrees, and `FunctionCall` prints each argument by index. `Literal` includes both value and runtime Python type names for clarity.

4.5 Parser and type checker boundary

A deliberate architectural boundary is maintained between syntax construction and semantic validation. The parser is responsible for producing a structurally valid AST with accurate source locations. The parse-tree renderer exposes the same parsed program in grammar-facing form, but it is not used for execution. Type correctness and semantic constraints are checked later by the type checker. For example, `_expr` may syntactically produce a node whose eventual type is not Boolean, but rejection of non-Boolean control-flow conditions is handled during semantic analysis. This separation keeps parser logic focused on grammar conformance and allows a clear responsibility split across team member contributions.

CHAPTER 5

TYPE INFERENCE ALGORITHM

5.1 Overview

Type inference in this project means that the programmer does not write explicit type declarations for variables or functions. Instead, the compiler infers types from assignments, literals, operations, function calls, and return statements. This keeps the language simple while still preserving static type safety.

The type checker runs after parsing. It receives the AST and validates the program before execution. Any inconsistent use of types is reported as a type error, and execution is stopped.

The interpreter stage does not repeat this validation: it walks the same AST with ordinary Python representations of values. Static type information remains in the symbol table produced by the checker; run-time values are not separately labelled with `LanguageType`, except that `print()` formats output by inspecting the value's Python type.

5.2 Type Environment

The type environment is represented by the symbol table. Conceptually, it is a mapping

$$\Gamma : \mathit{Identifier} \rightarrow \mathit{Type}$$

The environment grows as the checker visits the AST. On first assignment, a variable name is inserted with its inferred type. When a new block or a function call is entered, a child environment is created so that nested scopes can be checked independently while still accessing outer bindings when needed.

5.3 Type Inference Procedure

The type checker performs type inference by traversing the abstract syntax tree before interpretation. The algorithm is syntax-directed: each AST node is visited, the types of its child nodes are inferred first, and the current node is checked according to the language typing rules.

Algorithm TYPECHECK($node$, Γ)

Input: *node* = current AST node; Γ = current type environment / symbol table

Output: inferred type of *node*, or `TypeError`

1. **If** *node* is a **Literal**: return the corresponding primitive type: `Integer`, `Float`, `Boolean`, or `String`.
2. **If** *node* is an **Identifier**: look up the identifier in Γ. If it is not found, report an undefined variable error. Otherwise, return its stored type.
3. **If** *node* is a **BinaryOp**:
   - Infer the type of the left operand and the right operand.
   - If the operator is `+`, `-`, `*`, or `/`: require both operands to be `Integer`; return `Integer`.
   - If the operator is `+.`, `-.`, `*.`, or `/.`: require both operands to be `Float`; return `Float`.
   - If the operator is `==` or `!=`: require both operands to be the same arithmetic type; return `Boolean`.
   - Otherwise, report a `TypeError`.
4. **If** *node* is an **Assignment**:
   - Infer the type of the right-hand-side expression.
   - If the variable is not yet in Γ: insert it with the inferred type.
   - Else: verify the new type matches the existing type; if not, report a `TypeError`.
5. **If** *node* is an **If** statement:
   - Infer the condition type; require it to be `Boolean`.
   - Type-check the then-block in a child environment.
   - If an else-block exists, type-check it in a child environment.
6. **If** *node* is a **While** statement:
   - Infer the condition type; require it to be `Boolean`.
   - Type-check the loop body in a child environment.
7. **If** *node* is a **FunctionDef**:
   - Register the function name in Γ.
   - Parameter types are initially unknown.
   - Return type is inferred from `return` statements in the body.

8. **If** *node* is a **FunctionCall**:

   - If the function is the built-in `print`: require exactly one argument, infer that argument type, and return `Void`.
   - Otherwise, infer all argument types, validate them against the function signature, type-check the callee body if needed, and return the inferred function return type.

5.3.1 Example type inference trace

Table 5.1 traces the algorithm step by step for the program fragment

```
x = 3;
y = 4;
z = x + y;
```

showing how the type environment Γ grows as each sub-expression is inferred.

Table 5.1. Step-by-step type inference trace for `x = 3; y = 4; z = x + y;`

Step	Expression	Inferred type	Environment Γ
1	`3`	Integer	$\emptyset$
2	`x = 3`	Integer	$\{x{:}\mathit{Integer}\}$
3	`4`	Integer	$\{x{:}\mathit{Integer}\}$
4	`y = 4`	Integer	$\{x{:}\mathit{Integer},\ y{:}\mathit{Integer}\}$
5	`x + y`	Integer	unchanged
6	`z = x + y`	Integer	$\{x{:}\mathit{Integer},\ y{:}\mathit{Integer},\ z{:}\mathit{Integer}\}$

5.4 Inference Rules

5.4.1 Literals

Literal nodes have direct types:

$$\Gamma \vdash 42 : \texttt{Integer}$$

$$\Gamma \vdash 3.14 : \texttt{Float}$$

$$\Gamma \vdash \texttt{true} : \texttt{Boolean}$$

$$\Gamma \vdash \text{“}\texttt{hi}\text{”} : \texttt{String}$$

5.4.2 No Operator Overloading

The language does not support operator overloading. Following the Caml Light convention, integer and float arithmetic use entirely separate operator tokens, so every operator has a single, fixed type signature:

- `+`, `-`, `*`, `/` — require two `Integer` operands; result is `Integer`.
- `+.`, `-.`, `*.`, `/.` — require two `Float` operands; result is `Float`.
- `+` is not overloaded for string concatenation; applying it to a `String` is a type error.
- `==` and `!=` are not overloaded for `Boolean` or `String` equality.
- No implicit numeric promotion: `Integer + Float` is a type error.

Because every operator has a unique, unambiguous type derivable from its token alone, the compiler can always infer expression types without requiring explicit type declarations from the programmer. This design ensures that operator meaning is determined uniquely from the operator token itself, eliminating semantic ambiguity during type inference and simplifying static analysis.

5.4.3 Arithmetic Expressions

The integer operators `+`, `-`, `*`, and `/` require both operands to be `Integer` and always return `Integer` (including division, which uses integer floor division). The float operators `+.`, `-.`, `*.`, and `/.` require both operands to be `Float` and always return `Float`. Mixing an `Integer` and a `Float` with any operator is a type error.

Table 5.2. Arithmetic operator type rules

Left operand	Operator	Right operand	Result type
`Integer`	`+`, `-`, `*`, `/`	`Integer`	`Integer`
`Float`	`+.`, `-.`, `*.`, `/.`	`Float`	`Float`
`Integer`	any	`Float`	`TypeError`
`Float`	any	`Integer`	`TypeError`
Any	any	`String` or `Boolean`	`TypeError`

The following formal typing judgements capture these rules precisely. For integer operators ($\texttt{op} \in \{+, -, *, /\}$):

$$\frac{\Gamma \vdash e_1 : \mathit{Integer} \qquad \Gamma \vdash e_2 : \mathit{Integer}}{\Gamma \vdash e_1 \ \texttt{op} \ e_2 : \mathit{Integer}}$$

For float operators ($\texttt{op} \in \{+., -., *., /.\}$):

$$\frac{\Gamma \vdash e_1 : \mathit{Float} \qquad \Gamma \vdash e_2 : \mathit{Float}}{\Gamma \vdash e_1 \ \texttt{op} \ e_2 : \mathit{Float}}$$

For mixed-type operands with any operator (and symmetrically for *Float*/*Integer*):

$$\frac{\Gamma \vdash e_1 : \mathit{Integer} \qquad \Gamma \vdash e_2 : \mathit{Float}}{\text{TypeError}}$$

Note that each rule is keyed on the operator token, not on the operand types. This is only possible because integer and float arithmetic use distinct tokens — the property that eliminates operator overloading.

Unary negation

Unary integer negation (`-x`) is desugared by the parser to `0 - x`. Unary float negation (`-. x`) is desugared to `0.0 -. x`. The type checker therefore applies the standard binary rules in both cases.

5.4.4 Boolean Expressions

The comparison operators `==` and `!=` always produce `Boolean`. Both operators require two operands of the *same* arithmetic type — either both `Integer` or both `Float`; mixed-type comparisons such as `Integer == Float` are rejected. Comparisons involving `Boolean` or `String` operands are also rejected by the type checker. Although `Boolean` and `String` values exist in the language, equality and inequality are deliberately restricted to arithmetic expressions because the project specification defines basic Boolean expressions as equality and inequality between two arithmetic expressions.

Furthermore, the same-type requirement for comparisons — both operands must be either both `Integer` or both `Float` — is a direct consequence of the no-overloading design. If `==` accepted mixed `Integer` and `Float` operands, the compiler would need to determine whether to compare by promotion (implicitly converting `Integer` to `Float`) or by value identity, introducing the same ambiguity that the separate-operator design eliminates for arithmetic. Rejecting mixed-type comparisons ensures that `==` and `!=` each have exactly two valid type signatures — $(\mathit{Integer}, \mathit{Integer}) \to \mathit{Boolean}$ and $(\mathit{Float}, \mathit{Float}) \to \mathit{Boolean}$ — rather than a third mixed-type signature.

5.4.5 *Assignment Statement*

Assignments define variable types on first use. If a variable does not yet exist in the type environment, the checker infers the type of the right-hand side and inserts the variable into the environment. If the variable already exists, the new value must have the same type.

$$\text{If } x \notin \Gamma \text{ and } \Gamma \vdash e : T, \text{ then } \Gamma, x : T \vdash x = e$$

$$\text{If } x : T \in \Gamma \text{ and } \Gamma \vdash e : T, \text{ then } \Gamma \vdash x = e$$

5.4.6 *If-Then-Else*

The condition of an `if` statement must be `Boolean`. The then-block and else-block are checked in child scopes so that local operations can be validated cleanly.

$$\Gamma \vdash c : \texttt{Boolean} \quad \Gamma' \vdash B_{then} \quad \Gamma'' \vdash B_{else} \quad \Longrightarrow \quad \Gamma \vdash \texttt{if}\,(c)\,B_{then}\,\texttt{else}\,B_{else}$$

5.4.7 *While-Loop*

The condition of a `while` loop must also be `Boolean`. The loop body is checked in a child scope and must not violate the types of variables already introduced in the surrounding environment.

$$\Gamma \vdash c : \texttt{Boolean} \quad \Gamma' \vdash B \quad \Longrightarrow \quad \Gamma \vdash \texttt{while}\,(c)\,B$$

5.4.8 *Function Definition and Call*

Function parameter types are inferred from the first call to the function. After that first call, subsequent calls must use arguments of the same types. The return type is inferred from the `return` statements in the function body. If the function returns inconsistent types, the checker reports an error.

For functions that are defined but never called, the type checker performs a post-pass after all top-level statements have been processed. It infers parameter types by scanning the body for arithmetic and comparison usage, then checks the full body with those inferred types. This ensures that type errors inside never-called function bodies are still detected before execution.

Recursive calls are intentionally outside the scope of this implementation; supporting them would require additional fixed-point inference or explicit recursion handling that is orthogonal to the core type-inference algorithm.

Value parameter passing example

The language uses value parameter passing: each argument is evaluated in the caller's environment and a copy of its value is bound to the corresponding parameter in the callee's environment. Figure 5.1 shows the runtime environment before, during, and after a call to a function that mutates its parameter.

Source program:

```
x = 5;

def f(a) {
    a = a + 1;
}

f(x);
```

```
Before the call
Global scope:    x → 5

During the call
Global scope:  x → 5
Function scope (f):    a → 5, then a → 6

After the call
Global scope:    x → 5
```

Figure 5.1. Value parameter passing: the parameter `a` receives a copy of `x`, so mutating `a` inside `f` does not affect `x` in the caller.

Because the parameter receives a copy of the argument value rather than a shared reference, modifications to `a` inside the function do not affect `x` in the caller environment. This demonstrates standard value parameter passing semantics and avoids aliasing effects associated with reference-based mechanisms.

5.5 Type Error Reporting

Type errors are reported with source positions so that the user can locate the problem quickly. The format used by the system is:

```
[line X, col Y] TypeError:  message
```

Examples include assigning a `String` to an `Integer` variable, using a non-Boolean condition in an `if` statement, or supplying the wrong argument type to a function.

Table 5.3. Representative type errors detected by the static type checker

Invalid program	Reason	Detection stage
`x = 5; x = "hello";`	Inconsistent variable type	Type checker
`3 + 4.0`	Mixed arithmetic types	Type checker
`if (3) { ... }`	Condition not Boolean	Type checker
`print()`	Wrong argument count	Type checker
`x = print(5);`	Void assigned to variable	Type checker
`true == false`	Boolean comparison unsupported	Type checker

5.6 Example Type Inference Traces

Consider the following program fragment:

```
x = 3;
y = 4;
z = x + y;
print(z);
```

The checker infers:

- `x`: `Integer` (from literal `3`)
- `y`: `Integer` (from literal `4`)
- `x + y`: `Integer` (Integer + Integer → Integer)
- `z`: `Integer`

For a float example using dot-suffixed operators:

```
def addf(a, b) {
        return a +. b;
}

result = addf(2.0, 3.5);
```

The first function call infers `a`: `Float` and `b`: `Float`. The operator `+.` requires both operands to be `Float`, so the function return type is inferred as `Float`. The variable `result` is also inferred as `Float`.

CHAPTER 6

SYSTEM ARCHITECTURE AND IMPLEMENTATION

This chapter describes how the implementation is organised: the end-to-end processing flow, repository layout, technology choices, and the main components that realise static analysis and execution (type checker, interpreter, and user interfaces). Detailed treatment of lexical analysis, the symbol table, and parsing appears in Chapters 3 and 4; Chapter 5 presents the type-inference rules that the type checker implements.

6.1 Compiler pipeline

The system follows a pipeline with four processing stages and five labelled output sections exposed by the shared pipeline. Lexical and syntactic processing are kept separate from semantic analysis and execution, which simplifies testing and localises errors to the appropriate stage.

The overall flow of data through the system is shown in Figure 6.1.

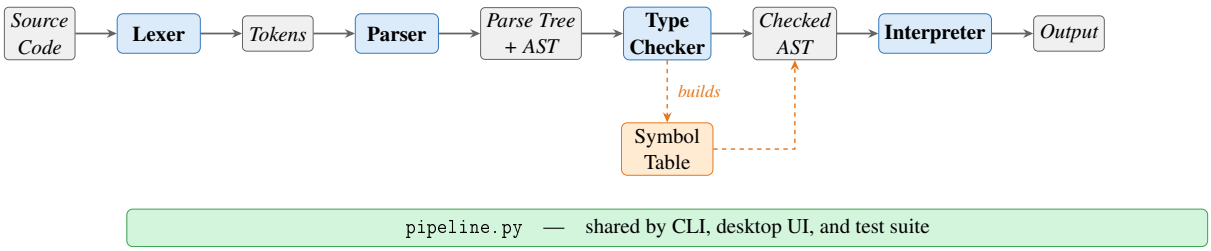


Figure 6.1. Front-end and interpreter pipeline overview.

Source characters are scanned by the lexer into a token stream. The parser consumes tokens and builds an AST that reflects program structure (expressions, statements, and control flow). The shared pipeline also renders a separate parse-tree view from the parsed program so that grammar structure can be inspected explicitly in the CLI, UI, and report. The type checker then walks the AST, populates and uses the symbol table, infers types, and validates scope and type rules. Only programs that pass the type checker proceed to interpretation. The interpreter evaluates the checked tree using ordinary Python values; it does not re-run the type rules, since correctness at runtime is guaranteed by the prior static pass. All stages are orchestrated through `pipeline.py`, which is shared by the command-line runner, the desktop UI, and the test suite.

6.1.1 End-to-end pipeline walkthrough

The following example traces the complete execution of a five-statement program through the exported pipeline views and execution stages, illustrating the correspondence between source text, tokens, parse tree, abstract syntax tree, inferred types, and runtime output.

Source program:

```
a = 3;
b = 4;
print(a + b);
c = 1.5;
print(c +. c);
```

Stage 1 — Lexer output (token stream):

```
  IDENTIFIER      'a'           line=1 col=1
  ASSIGN          '='           line=1 col=3
  INT_LITERAL     '3'           line=1 col=5
  SEMICOLON       ';'           line=1 col=6
  IDENTIFIER      'b'           line=2 col=1
  ASSIGN          '='           line=2 col=3
  INT_LITERAL     '4'           line=2 col=5
  SEMICOLON       ';'           line=2 col=6
  IDENTIFIER      'print'       line=3 col=1
  LPAREN          '('           line=3 col=6
  IDENTIFIER      'a'           line=3 col=7
  PLUS            '+'           line=3 col=9
  IDENTIFIER      'b'           line=3 col=11
  RPAREN          ')'           line=3 col=12
  SEMICOLON       ';'           line=3 col=13
  IDENTIFIER      'c'           line=4 col=1
  ASSIGN          '='           line=4 col=3
  FLOAT_LITERAL   '1.5'         line=4 col=5
  SEMICOLON       ';'           line=4 col=8
  IDENTIFIER      'print'       line=5 col=1
  LPAREN          '('           line=5 col=6
  IDENTIFIER      'c'           line=5 col=7
  PLUS_DOT        '+.'          line=5 col=9
  IDENTIFIER      'c'           line=5 col=12
  RPAREN          ')'           line=5 col=13
  SEMICOLON       ';'           line=5 col=14
  EOF             ''            line=5 col=15
```

Stage 2 — Parser output (ASCII branch parse tree):

```
program
|-- statement
|   `-- assignment
|       |-- IDENTIFIER(a)
|       |-- '='
|       |-- expr
```

```
|       |   `-- additive
|       |       `-- term
|       |           `-- factor
|       |               `-- INT_LITERAL(3)
|       `-- ';'
|-- statement
|   `-- assignment
|       |-- IDENTIFIER(b)
|       |-- '='
|       |-- expr
|       |   `-- additive
|       |       `-- term
|       |           `-- factor
|       |               `-- INT_LITERAL(4)
|       `-- ';'
|-- statement
|   `-- expr_stmt
|       |-- expr
|       |   `-- additive
|       |       `-- term
|       |           `-- factor
|       |               `-- function_call
|       |                   |-- IDENTIFIER(print)
|       |                   |-- '('
|       |                   |-- args
|       |                   |   `-- expr
|       |                   |       `-- additive
|       |                   |           |-- additive
|       |                   |           |   `-- term
|       |                   |           |       `-- factor
|       |                   |           |           `-- IDENTIFIER(a)
|       |                   |           |-- '+'
|       |                   |           `-- term
|       |                   |               `-- factor
|       |                   |                   `-- IDENTIFIER(b)
|       |                   `-- ')'
|       `-- ';'
|-- statement
|   `-- assignment
|       |-- IDENTIFIER(c)
|       |-- '='
|       |-- expr
|       |   `-- additive
|       |       `-- term
|       |           `-- factor
|       |               `-- FLOAT_LITERAL(1.5)
|       `-- ';'
|-- statement
|   `-- expr_stmt
|       |-- expr
|       |   `-- additive
|       |       `-- term
|       |           `-- factor
```

```
|       |                 `-- function_call
|       |                     |-- IDENTIFIER(print)
|       |                     |-- '('
|       |                     |-- args
|       |                     |   `-- expr
|       |                     |       `-- additive
|       |                     |           |-- additive
|       |                     |           |   `-- term
|       |                     |           |       `-- factor
|       |                     |           |           `-- IDENTIFIER(c)
|       |                     |           |-- '+.'
|       |                     |           `-- term
|       |                     |               `-- factor
|       |                     |                   `-- IDENTIFIER(c)
|       |                     `-- ')'
|       `-- ';'
`-- EOF
```

Stage 3 — Parser output (abstract syntax tree):

```
Program  [5 statement(s)]
    Assign  'a'  =
        Literal  3  (int)
    Assign  'b'  =
        Literal  4  (int)
    FunctionCall  'print'  [1 arg(s)]
        arg[0]:
            BinaryOp  '+'
                left:
                    Identifier  'a'
                right:
                    Identifier  'b'
    Assign  'c'  =
        Literal  1.5  (float)
    FunctionCall  'print'  [1 arg(s)]
        arg[0]:
            BinaryOp  '+.'
                left:
                    Identifier  'c'
                right:
                    Identifier  'c'
```

Stage 4 — Type checker output (inferred type environment):

Name	Inferred type
a	Integer
b	Integer
c	Float

The symbol table records only named symbols (a, b, and c). The type checker additionally

infers that the sub-expression `a + b` has type `Integer` (Integer + Integer → Integer) and that `c +. c` has type `Float` (Float +. Float → Float) during AST traversal, but sub-expression types are not stored in the symbol table.

Stage 5 — Interpreter output:

```
7 : Integer
3.0 : Float
```

The example demonstrates the full data flow: source characters are tokenised by the lexer, structured by the parser into both an inspectable parse tree and an executable AST, validated and annotated by the type checker, and finally evaluated by the interpreter to produce typed output. It also shows that the ASCII branch parse tree preserves the concrete operator spellings `'+'` and `'+.'`, making the distinction between integer and float arithmetic visible in the exported parser view.

Float operator walkthrough

To illustrate how the dot-suffixed float operators flow through each pipeline stage, consider a second example:

Source program:

```
x = 2.0;
y = 3.5;
print(x +. y);
```

Stage 1 — Lexer output (token stream):

```
  IDENTIFIER      'x'          line=1 col=1
  ASSIGN          '='          line=1 col=3
  FLOAT_LITERAL   '2.0'        line=1 col=5
  SEMICOLON       ';'          line=1 col=8
  IDENTIFIER      'y'          line=2 col=1
  ASSIGN          '='          line=2 col=3
  FLOAT_LITERAL   '3.5'        line=2 col=5
  SEMICOLON       ';'          line=2 col=8
  IDENTIFIER      'print'      line=3 col=1
  LPAREN          '('          line=3 col=6
  IDENTIFIER      'x'          line=3 col=7
  PLUS_DOT        '+.'         line=3 col=9
  IDENTIFIER      'y'          line=3 col=12
  RPAREN          ')'          line=3 col=13
  SEMICOLON       ';'          line=3 col=14
  EOF             ''           line=3 col=15
```

Note that the lexer emits `PLUS_DOT` (not `PLUS`) for `+.`, distinguishing float addition from integer

addition at the token level. Note also that `print` is tokenised as an ordinary `IDENTIFIER` — it is not a reserved keyword in this language.

Stage 3 — Parser output (abstract syntax tree):

```
Program  [3 statement(s)]
    Assign  'x'  =
        Literal  2.0  (float)
    Assign  'y'  =
        Literal  3.5  (float)
    FunctionCall  'print'  [1 arg(s)]
        arg[0]:
            BinaryOp  '+.'
                left:
                    Identifier  'x'
                right:
                    Identifier  'y'
```

The AST records the operator as `+.` rather than `+`, preserving the type distinction through to semantic analysis.

Stage 4 — Type checker output (inferred type environment):

Name	Inferred type
x	Float
y	Float

The type checker verifies that both operands of `+.` are `Float`. Had either operand been `Integer`, a `TypeError` would be reported before execution.

Stage 5 — Interpreter output:

```
5.5 : Float
```

This second walkthrough demonstrates that the dot-suffixed operators are visible at every pipeline stage — from the lexer token `PLUS_DOT` to the AST node `BinaryOp '+.'` to the final typed output — ensuring that integer and float arithmetic are kept strictly separate throughout the system.

6.1.2 Entry points and shared pipeline

All stages are orchestrated by `components/pipeline.py`, which is shared by the command-line runner, the desktop UI, and the test suite. From the project root, the CLI and desktop UI can be launched as follows:

```
# macOS / Linux
python3 main.py        # run CLI (built-in sample or script file)
python3 ui.py          # open desktop workbench

# Windows
python main.py
python ui.py
```

The CLI prints five labelled stages in order: tokens, parse tree, AST text, type table, and execution output. The desktop workbench uses the same pipeline internally, presenting each stage in a separate tab for inspection.

6.2 Project structure

The main source files and their responsibilities are summarised in Table 6.1.

Table 6.1. Main source files

File	Responsibility
`main.py`	Command-line runner for sample code or a script file
`ui.py`	Desktop UI for editing and running a script
`components/tokens.py`	Token classes and token categories
`components/lexica.py`	Lexical analysis
`components/ast_nodes.py`	AST node definitions
`components/parser.py`	Recursive-descent parser
`components/parse_tree_printer.py`	Parse-tree renderer for inspection
`components/ast_printer.py`	AST renderer for debugging and reporting
`components/symbol_table.py`	Scoped symbol table and semantic types
`components/type_checker.py`	Static type checker and inference
`components/interpreter.py`	Tree-walking interpreter
`components/pipeline.py`	Shared pipeline for CLI, UI, and tests

6.3 Technology stack

The project uses Python and standard-library tools only (Table 6.2).

Table 6.2. Technology stack

Tool	Purpose
Python 3.10+	Core implementation language
`dataclasses`	AST and runtime helper structures
`tkinter`	Desktop graphical interface
`unittest`	Automated testing

6.4 Type checker

The `TypeChecker` class implements the inference and checking rules described in Chapter 5. It traverses the AST before execution, uses the symbol table to record variable and function information, infers types on first assignment, validates arithmetic and comparison expressions, enforces Boolean conditions for control flow, infers function parameter types from the first call (or from body usage for uncalled functions), and infers function return types from `return` statements. Programs that fail these rules are rejected before interpretation.

6.4.1 Scope of the implemented type system

In scope terms, this component turns a syntactically valid AST into a semantically valid program:

- static typing rules for arithmetic, comparison, assignment, control flow, and functions,
- type checking for expressions and statements,
- variable type inference from first assignment,
- function parameter inference from the first call (or from body usage for uncalled functions),
- function return type inference from `return` statements,
- source-positioned type errors when a program violates a typing rule.

6.5 Interpreter

The interpreter is a tree-walking evaluator over the checked AST. It maintains nested runtime environments for variables and function calls. Assignments write through the current or nearest enclosing scope; `if` selects a branch; `while` repeats until its condition becomes false; function calls bind arguments by value and return through an internal return mechanism.

The built-in `print()` function outputs both value and runtime type. All four types are supported:

```
print(42);
print(3.14);
print(true);
print("plc");
```

These produce the output lines `42 : Integer`, `3.14 : Float`, `true : Boolean`, and `"plc" : String` respectively.

6.5.1 Scope of the implemented runtime

The runtime stage provides observable behaviour and integrates with the shared pipeline:

- assignment execution,
- `if` execution,
- `while` execution,
- function execution with value parameter passing,
- `print()` execution with immediate output,
- execution behind the same `run_pipeline` path used by the CLI, UI, and tests.

6.6 Graphical user interface

The desktop workbench (`ui.py`) is implemented with Python's `tkinter` standard library and requires no additional dependencies. The window is titled *125970-126690-126687-Ass2-PLC* and opens at 1200×760 pixels in a horizontal split layout.

Toolbar. A top toolbar provides three buttons — *Open Script*, *Run*, and *Clear Output* — and a right-aligned status label that reports *Ready* on launch, *Loaded* <*filename*> after a script is opened, *Ran* <*filename*> after a successful run on a loaded script file, *Ran editor contents* when running the editor buffer directly without having opened a file, *Run failed* when a pipeline stage raises an error, and *Output cleared* after the *Clear Output* button is pressed.

Left panel (source editor). A monospaced `Text` widget with both horizontal and vertical scrollbars allows the user to enter or paste arbitrary source code. The *Open Script* button populates the editor from a file. The editor is pre-loaded with the built-in sample program on first launch.

Right panel (output tabs). A `Notebook` widget presents six tabs, each backed by a scrollable read-only `Text` widget with both horizontal and vertical scrollbars:

- *Execution Output* — the `print()` values produced by stage 5, formatted as `value : Type`. The type label is determined at runtime by inspecting the Python type of the

evaluated value; because the type checker guarantees correctness before execution, this always agrees with the statically inferred type;

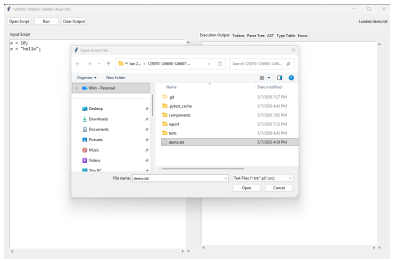
- *Tokens* — the token stream from stage 1, one token per line with type, lexeme, and source position;
- *Parse Tree* — a grammar-oriented ASCII branch rendering of stage 2 using non-terminal labels such as `statement`, `assignment`, `expr`, `comparison`, `additive`, `term`, and `factor`;
- *AST* — the indented executable tree from stage 3;
- *Type Table* — the symbol table from stage 4 showing each name, kind, inferred type, initialisation state, and parameter list;
- *Errors* — populated only when a pipeline stage fails; shows the error message (e.g. `[line 2, col 5] TypeError: ...`). On a successful run this tab is cleared automatically.

Error handling. If any pipeline stage raises an exception, the error message is written exclusively to the *Errors* tab and the remaining tabs are cleared. No modal dialogs are used, so the user can edit and rerun without dismissing a popup.

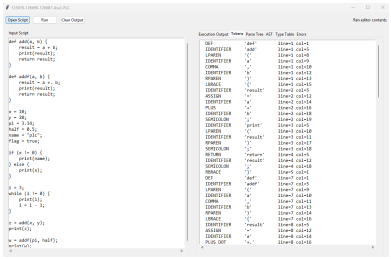
Keyboard shortcuts. `F5` and `Ctrl+Enter` are both bound to the *Run* action, allowing rapid edit-run cycles without reaching for the mouse.

6.6.1 Screenshots

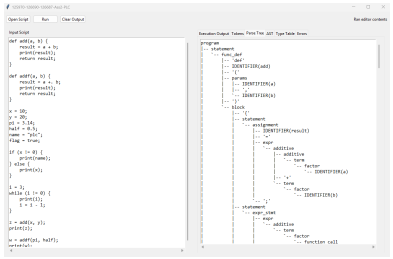
Figures 6.2 and 6.3 show the desktop workbench and CLI output.



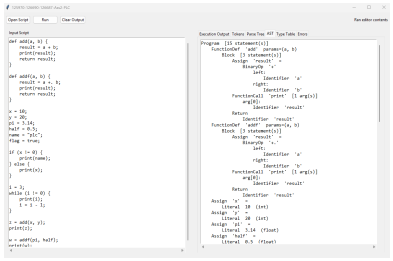
(a) Open script.



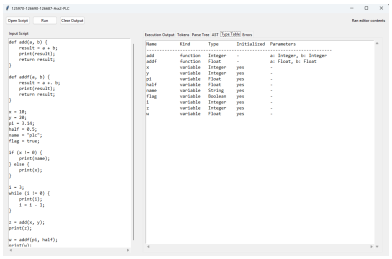
(b) Tokens.



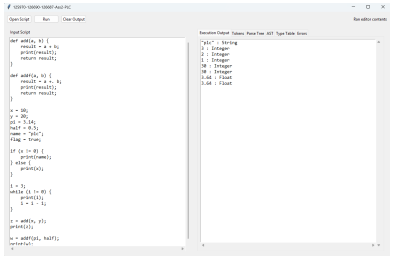
(c) Parse tree.



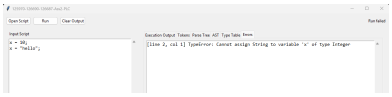
(d) AST.



(e) Types.



(f) Output.

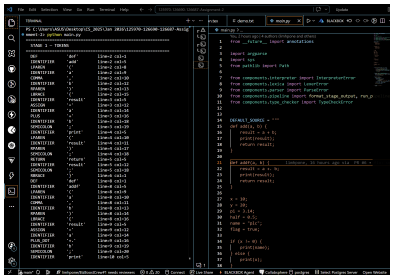


(g) Errors.

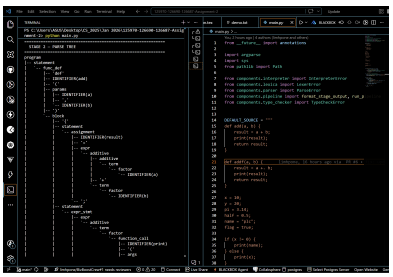
Figure 6.2. Desktop workbench tabs.

6.7 Command-line interface

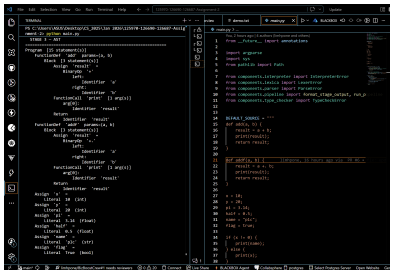
The CLI runner (`main.py`) accepts an optional script path; without one it runs the built-in sample and prints five labelled stages to `stdout`. Errors are reported cleanly to `stderr`: if the specified file does not exist, the runner prints `ERROR: File not found: <path>` and exits with code 1. If any pipeline stage raises a language error (`LexerError`, `ParseError`, `TypeCheckError`, or `InterpreterError`), the message is printed as `ERROR: [line X, col Y] <ErrorType>: <detail>` and the process exits with code 1. In both cases no Python traceback is shown to the user.



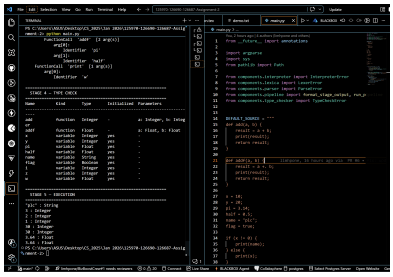
(a) Tokens.



(b) Parse tree.



(c) AST.



(d) Types and output.

(e) CLI error.

Figure 6.3. CLI stages and error handling.

CHAPTER 7

TESTING AND VERIFICATION

7.1 Manual Testing

Prior to the construction of the automated suite, each language feature was exercised manually by running the full pipeline and inspecting stage outputs. Table 7.1 summarises the categories covered.

Table 7.1. Representative manually tested categories

Category	Program Behaviour	Expected Result
Arithmetic	Evaluate integer and float expressions separately	Correct value and inferred type
If-then-else	Choose one branch from a Boolean condition	Only matching branch runs
While-loop	Print during repeated execution	One output line per iteration
Functions	Infer parameters and return type	Valid call and correct return value
Type mismatch	Reassign incompatible type	Type error before runtime

7.1.1 Lexer and Symbol Table

The lexer was verified manually by running the full pipeline and inspecting Stage 1 token output for programs that exercise every token category: integer and float literals, Boolean literals, string literals, the five reserved keywords, two-character comparison operators (`==`, `!=`), single-character operators, and delimiters. Identifier scanning with keyword fallback was confirmed by checking that `true` and `false` are emitted as `BOOL_LITERAL` tokens, the five reserved words (`if`, `else`, `while`, `def`, `return`) as their respective keyword token types, and ordinary names including `print` as `IDENTIFIER` tokens. For example, the source fragment `x = 10;` produces the token sequence:

```
IDENTIFIER  'x'    line=1 col=1
ASSIGN      '='    line=1 col=3
INT_LITERAL '10'   line=1 col=5
SEMICOLON   ';'    line=1 col=7
```

Error paths were also checked: supplying a bare `!` character raises a `LexerError` with a source position and a message suggesting `!=`, and an unterminated string literal raises a `LexerError` citing the opening line.

The symbol table was verified through the type checker and the Stage 3 output. Duplicate variable definitions were confirmed to raise a `SymbolTableError` before execution. Scope isolation was verified with nested blocks: a variable defined inside an `if` or `while` body is not visible in the outer scope after the block ends. The `format_table()` output was checked against the expected column layout (name, kind, type, initialisation status, parameters) for both variable and function symbols.

7.1.2 Parser, Parse Tree, and AST

The parser was verified manually by running the full pipeline and inspecting both the Stage 2 parse-tree output and the Stage 3 AST output. The following cases were checked:

- the parse-tree view contains grammar-oriented labels such as `program`, `statement`, `assignment`, and `expr`, confirming that the system now exposes a separate parse tree rather than only the executable AST,
- arithmetic expressions with mixed precedence (e.g. `x + y * 2`) produce a tree where multiplication is a deeper sub-tree than addition, confirming the `_expr`/`_additive`/`_term`/`_factor` precedence encoding,
- assignment versus expression statement dispatch: `x = 5;` produces an `Assign` node while `add(1,2);` produces a `FunctionCall` node at the statement level, confirming the IDENTIFIER+ASSIGN lookahead,
- `if` with and without an `else` clause: the resulting `If` node has a non-null `else_block` only when the clause is present,
- function definitions store only parameter names in the `FunctionDef` node; the AST printer output was inspected to confirm no type annotations are present at this stage,
- string literals in `Literal` nodes have their surrounding double-quotes stripped; `print("hello")` prints `"hello" : String` at runtime (the interpreter re-adds the quotes when formatting string output),

- parse errors: missing semicolons, unmatched parentheses, and unexpected tokens all produce `[line X, col Y] ParseError:` messages with correct source positions.

7.1.3 Arithmetic Expressions

Arithmetic expressions were tested with integer-only and float-only programs to verify operator precedence and strict type separation. The expression `x + y * 2` confirmed that multiplication binds more tightly than addition, and mixing integer and float operands correctly produced a type error, confirming the no-overloading design. The test suite explicitly verifies that mixed-type arithmetic is rejected:

- `1 + 2` — valid Integer arithmetic.
- `2.0 +. 3.5` — valid Float arithmetic.
- `5.0 -. 1.5`, `2.0 *. 3.0`, `9.0 /. 4.0` — valid Float arithmetic for the other dot-suffixed operators.
- `1 + 2.5` — `TypeError`: operator + requires two Integer operands.
- `2 +. 3` — `TypeError`: operator `+.` requires two Float operands.
- `10 / 0` and `10.0 /. 0.0` — `InterpreterError`: division by zero, raised at runtime.
- `1.0 +. 2.0 *. 3.0` — evaluates to `7.0` confirming `*.` binds more tightly than `+.`.

These rejection tests are not incidental — they are the primary evidence that the language satisfies the specification's no-overloading requirement. In a promotion-based language, the expressions `1 + 2.5` and `2 +. 3` would silently succeed (returning `3.5 : Float` and `5.0 : Float` respectively). The fact that both are rejected as `TypeErrors` demonstrates that each operator token has a single, fixed type signature and that no implicit type coercion occurs. The test suite contains 7 dedicated type-error tests for mixed-type and wrong-type operator usage (see Table 7.3, `TypeCheckerTests`), ensuring that the no-overloading guarantee is verified mechanically rather than by assertion alone.

7.1.4 Boolean Expressions

Boolean expressions were tested inside `if` and `while` conditions. The type checker correctly enforced that both operands of == and != must be arithmetic; supplying a `Boolean` or `String` operand produced a source-positioned `TypeCheckError` before any execution.

7.1.5 Assignment Statements

Assignments were tested for both first-use type inference and reassignment consistency. Assigning `5` to a variable and subsequently assigning `"hello"` to the same variable correctly produced a type error, confirming that the inferred type is fixed after first assignment.

7.1.6 If-Then-Else

Conditional programs were constructed to verify that exactly one branch executes for a given condition and that neither branch is entered when the condition evaluates to the opposite truth value. Non-Boolean conditions were also supplied and confirmed to be rejected by the type checker before execution.

7.1.7 While-Loop

While-loops were tested with an integer countdown from three to zero. The results confirmed that the loop body executes once per iteration and terminates when the condition becomes false, with `print()` producing one output line per pass rather than accumulating output until loop exit.

7.1.8 Functions

Function tests covered value parameter passing, local scope execution, and return value propagation. The type checker correctly inferred parameter types from the first call site and enforced that signature for all subsequent calls to the same function.

7.1.9 Print

The built-in `print()` function was tested with all supported runtime types. The output includes the printed value together with its type, such as `3.14 : Float` and `"plc" : String`.

7.1.10 Unary Minus

The unary minus operator was verified for integer literals (`x = -5`), float literals (`y = -. 3.14`), negated float variables (`y = -. x`), negated integer variables (`y = -x`), and negation inside a larger expression (`x = 3 + -2`). In each case the type checker assigned the correct type and the interpreter produced the expected value. Note that float negation uses the `-.` token, consistent with the Caml Light convention; using plain `-` on a `Float` operand is rejected

by the type checker.

7.2 Type Error Detection

The checker was verified with invalid programs involving incompatible assignment, invalid arithmetic operands, wrong function arguments, and non-Boolean conditions. In each case the system stopped before execution and reported a source-positioned type error. Representative error messages are shown below.

```
# Assignment type mismatch
[line 2, col 1] TypeError: Cannot assign String to variable 'x' of type
    Integer

# Non-Boolean if condition
[line 1, col 4] TypeError: If condition must have type Boolean

# Wrong argument type for function
[line 5, col 9] TypeError: Argument 1 of 'add' must be Integer, got
   String
```

Table 7.2. Representative negative semantic test cases

Program fragment	Expected result
`1 + 2.0`	TypeError
`x = 5; x = true;`	TypeError
`if (3) { }`	TypeError
`print(1, 2)`	TypeError
`"a" + "b"`	TypeError
`true == false`	TypeError

These negative cases provide direct evidence that the type system enforces the no-overloading and same-type-comparison rules, rather than relying on positive cases alone.

7.3 Automated Test Suite

The automated regression suite is located in `tests/test_pipeline.py` and uses Python's built-in `unittest` framework. The tests are organised into five classes, each targeting a distinct compiler stage.

Table 7.3. Automated test coverage by class

Test class	Stage covered	Tests
`LexerTests`	Tokenisation (`lexica.py`)	12
`SymbolTableTests`	Scoped symbol table (`symbol_table.py`)	7
`ParserTests`	Parsing, precedence, parse-tree/AST output	7
`TypeCheckerTests`	Static type inference (`type_checker.py`)	22
`IntegrationTests`	Full pipeline execution	27
	Total	**75**

LexerTests verify that each token category (integer, float, Boolean, string, keywords, identifiers, two-character operators, and the four float-arithmetic operators `+.` `-.` `*.` `/.`) is produced correctly, that float literals such as `3.14` are not confused with integer followed by a dot operator, that source line numbers are tracked across newlines, and that illegal characters and unterminated strings raise `LexerError`.

ParserTests verify structural correctness of parsing. The 7 tests cover missing-semicolon and unclosed-brace error reporting, integer operator precedence (`*` before `+`), float operator precedence (`*.` before `+.`), left-associativity of arithmetic operators, parenthesised expressions overriding precedence, and availability of both the parse-tree and AST renderings in the pipeline result.

SymbolTableTests exercise the symbol table directly, without going through the full pipeline. They cover variable definition and lookup, duplicate definition raising `SymbolTableError` for both variables and functions, parent-scope lookup through `child_scope()`, undefined-symbol lookup error, and the `format_table()` output for both variable and function symbol rows.

TypeCheckerTests verify static type inference and error detection. The 22 tests cover integer and float arithmetic (all four dot-suffixed operators `+.` `-.` `*.` `/.` each verified to produce `Float`), integer floor division producing `Integer`, strict type separation (mixed Integer/Float rejected for both arithmetic and comparison operators, and integers used with a dot operator rejected), string and boolean type inference, assignment type mismatch, non-Boolean conditions, undefined variables, wrong argument count and type, and two tests for uncalled-function body checking: one confirms that a type error inside a never-called function body is detected, and another confirms that a valid uncalled function raises no error.

IntegrationTests run programs through all four pipeline stages and verify the final output. Cases include unary negation on all numeric types (including `-.` applied to a float variable),

division by zero for both `/` and `/.` (each must raise `InterpreterError`), both branches of `if/else`, while-loop countdown, `print()` with every type, value-parameter isolation, nested function calls, comparison results assigned to variables and printed directly, variable reassignment, and scope isolation (variables declared inside an `if` block are not visible outside it).

The tests can be run with:

```
# macOS / Linux
python3 -m unittest discover -s tests -v

# Windows
python -m unittest discover -s tests -v
```

All 75 tests pass.

7.4 Error Handling

The language system reports errors at four stages:

- lexical errors for malformed characters or unterminated strings,
- parse errors for invalid syntax such as missing semicolons,
- type errors for programs that violate static typing rules,
- runtime errors for division by zero and other execution failures.

All error messages include a source position in the form `[line X, col Y]` to aid debugging.

7.5 Limitations and Future Work

The implemented language intentionally restricts several advanced programming-language features in order to focus on core compiler construction principles and static type inference.

The current implementation does not support recursive functions, arrays, records, polymorphic typing, closures, or garbage collection. Type inference is syntax-directed and deterministic rather than based on full unification algorithms such as Hindley–Milner inference. The execution model is interpreted directly from the AST rather than compiled into bytecode or machine code.

Future extensions may include recursive function support, richer data structures, additional control-flow constructs, optimisation passes, and compilation to a lower-level intermediate representation.

CHAPTER 8

CONCLUSION

This project delivered a complete, statically-typed interpreted language supporting Integer, Float, Boolean, and String types, with arithmetic, comparisons, assignments, if-then-else, while-loops, functions with value parameter passing, and a built-in `print()` function.

The system follows a four-stage pipeline: lexer, parser, type checker, and interpreter. Variable and function types are inferred without explicit declarations; programs that violate type rules are rejected before execution. For uncalled functions, the type checker performs a post-pass to infer parameter types from body usage, ensuring type errors are always caught. A central design decision — following the Caml Light convention of using separate operator tokens for integer and float arithmetic — ensures that the specification's no-overloading requirement is satisfied in the strongest possible sense: every operator token carries exactly one type signature, and the compiler never inspects operand types to resolve operator meaning. This eliminates not only traditional overloading but also the implicit numeric promotion found in many mainstream languages, resulting in fully deterministic type inference where every expression's type is derivable from the operator tokens and literal forms alone.

To support clearer correspondence between the formal grammar and accepted programs, the parser stage now exposes both a grammar-oriented parse-tree view and the executable AST used by later stages. The built-in `print()` operation is now implemented as a normal callable built-in rather than a dedicated statement form. The 75 automated tests across five classes — lexical analysis, symbol table, parsing, type checking, and full pipeline — confirm correct behaviour for all language features. Both a command-line runner and a desktop UI are provided.

The project demonstrates how lexical analysis, parsing, semantic analysis, type inference, and interpretation can be integrated into a coherent compiler pipeline with clear separation of responsibilities between stages. The implementation prioritised semantic clarity, deterministic type inference, and modular compiler structure over advanced language features. Future work could extend the language with recursive functions, richer data structures, and code generation.

REFERENCES

Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, Boston, MA, 2nd edition, 2006.

Robert Nystrom. *Crafting Interpreters*. Genever Benning, Seattle, WA, 2021.

Python Software Foundation. Python documentation: dataclasses — data classes. `https://docs.python.org/3/library/dataclasses.html`, 2026a. Accessed: April 2026.

Python Software Foundation. Python documentation: Tkinter — python interface to tcl/tk. `https://docs.python.org/3/library/tkinter.html`, 2026b. Accessed: April 2026.

Python Software Foundation. Python documentation: unittest — unit testing framework. `https://docs.python.org/3/library/unittest.html`, 2026c. Accessed: April 2026.

APPENDIX A: SOURCE CODE

The complete source code is available on GitHub:

https://github.com/The-Token-Trio/125970-126690-126687-Assignment2-PLC

A video walkthrough of the CLI and GUI is available at:

https://youtu.be/sR2JRm6eYzM

The project can be run from the command line or the desktop UI:

```
# macOS / Linux
python3 main.py
python3 ui.py
python3 -m unittest discover -s tests -v

# Windows
python main.py
python ui.py
python -m unittest discover -s tests -v
```

No external Python dependencies are required.

APPENDIX B: TEAM MEMBERS AND CONTRIBUTIONS

Name	Student ID	Contributions
Aye Khin Khin Hpone (Yolanda Lim)	st125970	Static typing and type checking; assignment, if, while, function, and `print()` execution; unary minus inference/execution; pipeline integration; parse-tree/AST exposure; automated testing (75 tests)
Applegate T. Tun Oo	st126690	Lexer implementation (scanning, tokenisation, line/column tracking, error reporting); token definitions; keywords, operators, identifiers, and literals; symbol table (variable/function/type storage)
Win Htut Naing	st126687	Arithmetic and boolean expressions; assignment, if-then-else, while-loop, function definition/call, and `print()` parsing; unary minus parsing

A.1 Entry Points

```
from __future__ import annotations

import argparse
import sys
from pathlib import Path

from components.interpreter import InterpreterError
from components.lexica import LexerError
from components.parser import ParseError
from components.pipeline import format_stage_output, run_pipeline
from components.type_checker import TypeCheckError


DEFAULT_SOURCE = """
def add(a, b) {
    result = a + b;
    print(result);
    return result;
}

def addf(a, b) {
    result = a +. b;
    print(result);
    return result;
}

x = 10;
y = 20;
pi = 3.14;
half = 0.5;
name = "plc";
flag = true;

if (x != 0) {
    print(name);
} else {
    print(x);
}

i = 3;
while (i != 0) {
    print(i);
    i = i - 1;
}

z = add(x, y);
print(z);

w = addf(pi, half);
print(w);
"""
```

```
52
53
54  def run_source(source_code: str) -> None:
55      try:
56          result = run_pipeline(source_code)
57          print(format_stage_output(result))
58      except (LexerError, ParseError, TypeCheckError, InterpreterError)
            as error:
59          print(f"ERROR: {error}", file=sys.stderr)
60          sys.exit(1)
61
62
63  def _parse_args() -> argparse.Namespace:
64      parser = argparse.ArgumentParser(description="Run the programming
            language pipeline.")
65      parser.add_argument("script", nargs="?", help="Optional path to a
            source file to run")
66      return parser.parse_args()
67
68
69  def main() -> None:
70      args = _parse_args()
71      if args.script:
72          path = Path(args.script)
73          if not path.exists():
74              print(f"ERROR: File not found: {args.script}", file=sys.
                    stderr)
75              sys.exit(1)
76          source_code = path.read_text(encoding="utf-8")
77      else:
78          source_code = DEFAULT_SOURCE
79      run_source(source_code)
80
81
82  if __name__ == "__main__":
83      main()
```

Listing 8.1: main.py — command-line entry point

```
1   from __future__ import annotations
2
3   import tkinter as tk
4   from pathlib import Path
5   from tkinter import filedialog, ttk
6
7   from components.pipeline import format_tokens, run_pipeline
8
9
10  DEFAULT_SOURCE = """def add(a, b) {
11      result = a + b;
12      print(result);
13      return result;
14  }
```

```
def addf(a, b) {
    result = a +. b;
    print(result);
    return result;
}

x = 10;
y = 20;
pi = 3.14;
half = 0.5;
name = \"plc\";
flag = true;

if (x != 0) {
    print(name);
} else {
    print(x);
}

i = 3;
while (i != 0) {
    print(i);
    i = i - 1;
}

z = add(x, y);
print(z);

w = addf(pi, half);
print(w);
"""


class LanguageWorkbench:
    def __init__(self) -> None:
        self.root = tk.Tk()
        self.root.title("125970-126690-126687-Ass2-PLC")
        self.root.geometry("1200x760")
        self.current_file: Path | None = None

        self._build_layout()
        self.source_text.insert("1.0", DEFAULT_SOURCE)
        self.root.bind("<F5>", lambda _e: self._run_source())
        self.root.bind("<Control-Return>", lambda _e: self._run_source
            ())

    def _build_layout(self) -> None:
        toolbar = ttk.Frame(self.root, padding=10)
        toolbar.pack(fill=tk.X)

        ttk.Button(toolbar, text="Open Script", command=self.
            _open_script).pack(side=tk.LEFT)
```

```
66          ttk.Button(toolbar, text="Run", command=self._run_source).pack(
                side=tk.LEFT, padx=(8, 0))
67          ttk.Button(toolbar, text="Clear Output", command=self.
                _clear_output).pack(side=tk.LEFT, padx=(8, 0))
68
69          self.status_var = tk.StringVar(value="Ready")
70          ttk.Label(toolbar, textvariable=self.status_var).pack(side=tk.
                RIGHT)
71
72          body = ttk.Panedwindow(self.root, orient=tk.HORIZONTAL)
73          body.pack(fill=tk.BOTH, expand=True, padx=10, pady=(0, 10))
74
75          left_panel = ttk.Frame(body, padding=8)
76          right_panel = ttk.Frame(body, padding=8)
77          body.add(left_panel, weight=1)
78          body.add(right_panel, weight=1)
79
80          ttk.Label(left_panel, text="Input Script").pack(anchor=tk.W)
81          src_frame = ttk.Frame(left_panel)
82          src_frame.pack(fill=tk.BOTH, expand=True, pady=(6, 0))
83          self.source_text = tk.Text(src_frame, wrap=tk.NONE, font=("
                Consolas", 11))
84          src_vsb = ttk.Scrollbar(src_frame, orient=tk.VERTICAL, command=
                self.source_text.yview)
85          src_hsb = ttk.Scrollbar(src_frame, orient=tk.HORIZONTAL,
                command=self.source_text.xview)
86          self.source_text.configure(yscrollcommand=src_vsb.set,
                xscrollcommand=src_hsb.set)
87          src_vsb.pack(side=tk.RIGHT, fill=tk.Y)
88          src_hsb.pack(side=tk.BOTTOM, fill=tk.X)
89          self.source_text.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
90
91          notebook = ttk.Notebook(right_panel)
92          notebook.pack(fill=tk.BOTH, expand=True)
93
94          self.output_text = self._make_output_tab(notebook, "Execution
                Output")
95          self.tokens_text = self._make_output_tab(notebook, "Tokens")
96          self.parse_tree_text = self._make_output_tab(notebook, "Parse
                Tree")
97          self.ast_text = self._make_output_tab(notebook, "AST")
98          self.types_text = self._make_output_tab(notebook, "Type Table")
99          self.error_text = self._make_output_tab(notebook, "Errors")
100
101     def _make_output_tab(self, notebook: ttk.Notebook, title: str) ->
            tk.Text:
102         frame = ttk.Frame(notebook, padding=8)
103         notebook.add(frame, text=title)
104         text = tk.Text(frame, wrap=tk.NONE, font=("Consolas", 11),
                state=tk.DISABLED)
105         vsb = ttk.Scrollbar(frame, orient=tk.VERTICAL, command=text.
                yview)
106         hsb = ttk.Scrollbar(frame, orient=tk.HORIZONTAL, command=text.
```

```
            xview)
        text.configure(yscrollcommand=vsb.set, xscrollcommand=hsb.set)
        vsb.pack(side=tk.RIGHT, fill=tk.Y)
        hsb.pack(side=tk.BOTTOM, fill=tk.X)
        text.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
        return text

    def _open_script(self) -> None:
        file_path = filedialog.askopenfilename(
            title="Open Script File",
            filetypes=[("Text Files", "*.txt *.pl *.src"), ("All Files"
                , "*.*")],
        )
        if not file_path:
            return

        path = Path(file_path)
        self.current_file = path
        self.source_text.delete("1.0", tk.END)
        self.source_text.insert("1.0", path.read_text(encoding="utf-8")
            )
        self.status_var.set(f"Loaded {path.name}")

    def _run_source(self) -> None:
        source = self.source_text.get("1.0", tk.END)
        try:
            result = run_pipeline(source)
        except Exception as error:
            self._write_text(self.error_text, str(error))
            self._write_text(self.output_text, "")
            self._write_text(self.tokens_text, "")
            self._write_text(self.parse_tree_text, "")
            self._write_text(self.ast_text, "")
            self._write_text(self.types_text, "")
            self.status_var.set("Run failed")
            return

        self._write_text(self.output_text, "\n".join(result.outputs) if
            result.outputs else "(no output)")
        self._write_text(self.tokens_text, format_tokens(result.tokens)
            )
        self._write_text(self.parse_tree_text, result.parse_tree_text)
        self._write_text(self.ast_text, result.ast_text)
        self._write_text(self.types_text, result.checked_scope.
            format_table())
        self._write_text(self.error_text, "")

        current_label = self.current_file.name if self.current_file is
            not None else "editor contents"
        self.status_var.set(f"Ran {current_label}")

    def _clear_output(self) -> None:
        self._write_text(self.output_text, "")
```

```
        self._write_text(self.tokens_text, "")
        self._write_text(self.parse_tree_text, "")
        self._write_text(self.ast_text, "")
        self._write_text(self.types_text, "")
        self._write_text(self.error_text, "")
        self.status_var.set("Output cleared")

    @staticmethod
    def _write_text(widget: tk.Text, content: str) -> None:
        widget.config(state=tk.NORMAL)
        widget.delete("1.0", tk.END)
        widget.insert("1.0", content)
        widget.config(state=tk.DISABLED)

    def run(self) -> None:
        self.root.mainloop()


if __name__ == "__main__":
    LanguageWorkbench().run()
```

Listing 8.2: ui.py — Tkinter desktop workbench

A.2 Core Components

```
from __future__ import annotations

from dataclasses import dataclass
from enum import Enum, auto


class TokenType(Enum):
    # Literals
    INT_LITERAL = auto()
    FLOAT_LITERAL = auto()
    BOOL_LITERAL = auto()
    STRING_LITERAL = auto()

    # Identifier
    IDENTIFIER = auto()

    # Keywords
    IF = auto()
    ELSE = auto()
    WHILE = auto()
    DEF = auto()
    RETURN = auto()

    # Operators
    PLUS = auto()
    MINUS = auto()
    STAR = auto()
```

```
    SLASH = auto()
    PLUS_DOT = auto()     # +.  (float addition)
    MINUS_DOT = auto()    # -.  (float subtraction)
    STAR_DOT = auto()     # *.  (float multiplication)
    SLASH_DOT = auto()    # /.  (float division)
    EQUAL_EQUAL = auto()
    BANG_EQUAL = auto()
    ASSIGN = auto()

    # Delimiters
    LPAREN = auto()
    RPAREN = auto()
    LBRACE = auto()
    RBRACE = auto()
    COMMA = auto()
    SEMICOLON = auto()

    EOF = auto()


@dataclass(frozen=True)
class Token:
    token_type: TokenType
    lexeme: str
    line: int
    column: int
```

Listing 8.3: components/tokens.py — token type enumeration and Token dataclass

```
from __future__ import annotations

from typing import Iterator

from components.tokens import Token, TokenType


KEYWORDS: dict[str, TokenType] = {
    "if": TokenType.IF,
    "else": TokenType.ELSE,
    "while": TokenType.WHILE,
    "def": TokenType.DEF,
    "return": TokenType.RETURN,
    "true": TokenType.BOOL_LITERAL,
    "false": TokenType.BOOL_LITERAL,
}


# Arithmetic operators that may have a trailing '.' for the float
    variant
_ARITH_DOT: dict[str, tuple[TokenType, TokenType]] = {
    "+": (TokenType.PLUS,       TokenType.PLUS_DOT),
    "-": (TokenType.MINUS,      TokenType.MINUS_DOT),
    "*": (TokenType.STAR,       TokenType.STAR_DOT),
```

```
    "/": (TokenType.SLASH,      TokenType.SLASH_DOT),
}

SINGLE_CHAR_TOKENS: dict[str, TokenType] = {
    "=": TokenType.ASSIGN,
    "(": TokenType.LPAREN,
    ")": TokenType.RPAREN,
    "{": TokenType.LBRACE,
    "}": TokenType.RBRACE,
    ",": TokenType.COMMA,
    ";": TokenType.SEMICOLON,
}


class LexerError(Exception):
    pass


class Lexer:
    def __init__(self, source: str) -> None:
        self.source = source
        self.start = 0
        self.current = 0
        self.line = 1
        self.column = 1
        self.token_column = 1

    def tokenize(self) -> list[Token]:
        tokens: list[Token] = []
        while not self._is_at_end():
            self.start = self.current
            self.token_column = self.column
            token = self._scan_token()
            if token is not None:
                tokens.append(token)
        tokens.append(Token(TokenType.EOF, "", self.line, self.column))
        return tokens

    def _scan_token(self) -> Token | None:
        char = self._advance()

        # Whitespace handling
        if char in {" ", "\t", "\r"}:
            return None
        if char == "\n":
            self.line += 1
            self.column = 1
            return None

        # Two-char operators
        if char == "=":
            if self._match("="):
                return self._make_token(TokenType.EQUAL_EQUAL)
```

```
            return self._make_token(TokenType.ASSIGN)
        if char == "!":
            if self._match("="):
                return self._make_token(TokenType.BANG_EQUAL)
            raise LexerError(self._error_message("Unexpected '!'. Did you mean '!='?"))

        # Arithmetic operators: + - * /  (with optional trailing '.' for float variant)
        arith = _ARITH_DOT.get(char)
        if arith is not None:
            int_tok, float_tok = arith
            # Unary-minus lookahead: '-' followed by a digit is NOT '-.' even if
            # next char is '.' -- we still need to check char after '.' is not digit
            # (to avoid consuming the '.' of a float literal like 3.14).
            # Rule: X. is a float operator only when the char after '.' is NOT a digit.
            if self._peek() == "." and not self._peek_next().isdigit():
                self._advance()  # consume '.'
                return self._make_token(float_tok)
            return self._make_token(int_tok)

        # Single-char operators/delimiters
        single_char_token = SINGLE_CHAR_TOKENS.get(char)
        if single_char_token is not None:
            return self._make_token(single_char_token)

        if char == '"':
            return self._string()

        if char.isdigit():
            return self._number()

        if self._is_identifier_start(char):
            return self._identifier()

        raise LexerError(self._error_message(f"Unexpected character: {char!r}"))

    def _string(self) -> Token:
        while self._peek() != '"' and not self._is_at_end():
            if self._peek() == "\n":
                self.line += 1
                self.column = 1
            self._advance()

        if self._is_at_end():
            raise LexerError(self._error_message("Unterminated string literal"))

```

```
        # Closing quote
        self._advance()
        return self._make_token(TokenType.STRING_LITERAL)

    def _number(self) -> Token:
        while self._peek().isdigit():
            self._advance()

        is_float = False
        if self._peek() == "." and self._peek_next().isdigit():
            is_float = True
            self._advance()
            while self._peek().isdigit():
                self._advance()

        if is_float:
            return self._make_token(TokenType.FLOAT_LITERAL)
        return self._make_token(TokenType.INT_LITERAL)

    def _identifier(self) -> Token:
        while self._is_identifier_part(self._peek()):
            self._advance()

        lexeme = self._current_lexeme()
        token_type = KEYWORDS.get(lexeme, TokenType.IDENTIFIER)
        return self._make_token(token_type)

    def _current_lexeme(self) -> str:
        return self.source[self.start : self.current]

    def _make_token(self, token_type: TokenType) -> Token:
        return Token(token_type, self._current_lexeme(), self.line,
           self.token_column)

    def _advance(self) -> str:
        char = self.source[self.current]
        self.current += 1
        self.column += 1
        return char

    def _match(self, expected: str) -> bool:
        if self._is_at_end():
            return False
        if self.source[self.current] != expected:
            return False

        self.current += 1
        self.column += 1
        return True

    def _peek(self) -> str:
        if self._is_at_end():
            return "\0"
```

```
        return self.source[self.current]

    def _peek_next(self) -> str:
        if self.current + 1 >= len(self.source):
            return "\0"
        return self.source[self.current + 1]

    def _is_at_end(self) -> bool:
        return self.current >= len(self.source)

    @staticmethod
    def _is_identifier_start(char: str) -> bool:
        return char.isalpha() or char == "_"

    @staticmethod
    def _is_identifier_part(char: str) -> bool:
        return char.isalnum() or char == "_"

    def _error_message(self, message: str) -> str:
        return f"[line {self.line}, col {self.token_column}] LexerError: {message}"


def iter_tokens(source: str) -> Iterator[Token]:
    yield from Lexer(source).tokenize()
```

Listing 8.4: components/lexica.py — lexical analyser

```
from __future__ import annotations

from dataclasses import dataclass, field
from typing import Optional


# ---------------------------------------------------------------------------
# Base
# ---------------------------------------------------------------------------


@dataclass
class ASTNode:
    """Base class for every AST node. Carries source location for diagnostics."""
    line: int = field(default=0, kw_only=True)
    column: int = field(default=0, kw_only=True)


# ---------------------------------------------------------------------------
```

```python
# Expressions
#   ------------------------------------------------------------------------


@dataclass
class Literal(ASTNode):
    """An integer, float, boolean, or string literal.

    Examples:  42   3.14   true   "hello"
    """
    value: int | float | bool | str


@dataclass
class Identifier(ASTNode):
    """A variable or function name used as an expression.

    Example:  x
    """
    name: str


@dataclass
class BinaryOp(ASTNode):
    """An arithmetic or comparison binary operation.

    op is one of: '+' '-' '*' '/' '+.' '-.' '*.' '/.' '==' '!='
    Example:  a + b   x == 0   a +. b
    """
    op: str
    left: ASTNode
    right: ASTNode


@dataclass
class FunctionCall(ASTNode):
    """A call to a user-defined function.

    Example:  add(1, 2)
    """
    name: str
    args: list[ASTNode] = field(default_factory=list)


#   ------------------------------------------------------------------------

# Statements
#   ------------------------------------------------------------------------


```

```
@dataclass
class Assign(ASTNode):
    """Variable assignment (first use infers type; later uses must
       match).

    Example:  x = 10;
    """
    name: str
    value: ASTNode


@dataclass
class Return(ASTNode):
    """Return statement inside a function body.

    Example:  return result;
    """
    expr: ASTNode


@dataclass
class Block(ASTNode):
    """A brace-enclosed sequence of statements.

    Example:  { x = 1; print(x); }
    """
    statements: list[ASTNode] = field(default_factory=list)


@dataclass
class If(ASTNode):
    """If / optional else control flow.

    Example:  if (x != 0) { print(x); } else { print(0); }
    """
    condition: ASTNode
    then_block: Block
    else_block: Optional[Block]


@dataclass
class While(ASTNode):
    """While loop.

    Example:  while (x != 0) { x = x - 1; }
    """
    condition: ASTNode
    body: Block


@dataclass
class FunctionDef(ASTNode):
    """Function definition.
```

```
118
119     Example:  def add(a, b) { return a + b; }
120
121     Note: parameter types are unknown at parse time -- Member 3's type
            checker
122     will infer them from usage. We store only the parameter names here.
123     """
124     name: str
125     params: list[str]
126     body: Block
127
128
129 #
        ------------------------------------------------------------------------

130 # Top-level
131 #
        ------------------------------------------------------------------------

132
133 @dataclass
134 class Program(ASTNode):
135     """Root node: the entire source file as a list of statements."""
136     statements: list[ASTNode] = field(default_factory=list)
```

Listing 8.5: components/ast_nodes.py — AST node definitions

```
1  from __future__ import annotations
2
3  from typing import Optional
4
5  from components.tokens import Token, TokenType
6  from components.ast_nodes import (
7      ASTNode,
8      Program,
9      Block,
10     Assign,
11     If,
12     While,
13     FunctionDef,
14     FunctionCall,
15     Return,
16     BinaryOp,
17     Literal,
18     Identifier,
19 )
20
21
22 #
       ------------------------------------------------------------------------

23 # Parser error
24 #
```

```
    # ------------------------------------------------------------------------


class ParseError(Exception):
    pass


#
    # ------------------------------------------------------------------------
# Parser
#
    # ------------------------------------------------------------------------


class Parser:
    """Recursive-descent parser.

    Usage:
        tokens = Lexer(source).tokenize()
        tree   = Parser(tokens).parse()   # returns Program node
    """

    def __init__(self, tokens: list[Token]) -> None:
        self._tokens = tokens
        self._pos = 0

    #
        # ------------------------------------------------------------------------
    # Public entry point
    #
        # ------------------------------------------------------------------------


    def parse(self) -> Program:
        """Parse the full token stream and return the root Program node
            ."""
        line, col = self._peek().line, self._peek().column
        statements: list[ASTNode] = []
        while not self._is_at_end():
            statements.append(self._statement())
        return Program(statements=statements, line=line, column=col)

    #
        # ------------------------------------------------------------------------
    # Statements
    #
        # ------------------------------------------------------------------------


    def _statement(self) -> ASTNode:
```

```
        """Dispatch to the correct statement parser based on the
           current token."""
        tok = self._peek()

        if tok.token_type == TokenType.DEF:
            return self._func_def()

        if tok.token_type == TokenType.IF:
            return self._if_stmt()

        if tok.token_type == TokenType.WHILE:
            return self._while_stmt()

        if tok.token_type == TokenType.RETURN:
            return self._return_stmt()

        # assignment: IDENTIFIER "=" expr ";"
        if (
            tok.token_type == TokenType.IDENTIFIER
            and self._peek_next().token_type == TokenType.ASSIGN
        ):
            return self._assignment()

        # fallback: bare expression statement  expr ";"
        node = self._expr()
        self._expect(TokenType.SEMICOLON, "Expected ';' after
           expression")
        return node

    def _assignment(self) -> Assign:
        """IDENTIFIER '=' expr ';'"""
        name_tok = self._advance()                                  #
           IDENTIFIER
        self._advance()                                             # '='
        value = self._expr()
        self._expect(TokenType.SEMICOLON, "Expected ';' after
           assignment")
        return Assign(name=name_tok.lexeme, value=value,
                      line=name_tok.line, column=name_tok.column)

    def _if_stmt(self) -> If:
        """'if' '(' expr ')' block ( 'else' block )?"""
        tok = self._advance()                                       # 'if'
        self._expect(TokenType.LPAREN, "Expected '(' after 'if'")
        condition = self._expr()
        self._expect(TokenType.RPAREN, "Expected ')' after if condition
           ")
        then_block = self._block()
        else_block: Optional[Block] = None
        if self._check(TokenType.ELSE):
            self._advance()                                         # 'else
               '
            else_block = self._block()
```

```
        return If(condition=condition, then_block=then_block,
            else_block=else_block,
                  line=tok.line, column=tok.column)

    def _while_stmt(self) -> While:
        """'while' '(' expr ')' block"""
        tok = self._advance()                                    # '
            while'
        self._expect(TokenType.LPAREN, "Expected '(' after 'while'")
        condition = self._expr()
        self._expect(TokenType.RPAREN, "Expected ')' after while
            condition")
        body = self._block()
        return While(condition=condition, body=body,
                     line=tok.line, column=tok.column)

    def _func_def(self) -> FunctionDef:
        """'def' IDENTIFIER '(' params? ')' block"""
        tok = self._advance()                                    # 'def'
        name_tok = self._expect(TokenType.IDENTIFIER, "Expected
            function name after 'def'")
        self._expect(TokenType.LPAREN, "Expected '(' after function
            name")
        params = self._params()
        self._expect(TokenType.RPAREN, "Expected ')' after parameters")
        body = self._block()
        return FunctionDef(name=name_tok.lexeme, params=params, body=
            body,
                           line=tok.line, column=tok.column)

    def _return_stmt(self) -> Return:
        """'return' expr ';'"""
        tok = self._advance()                                    # '
            return'
        expr = self._expr()
        self._expect(TokenType.SEMICOLON, "Expected ';' after return
            value")
        return Return(expr=expr, line=tok.line, column=tok.column)

    def _block(self) -> Block:
        """'{' statement* '}'"""
        tok = self._expect(TokenType.LBRACE, "Expected '{'")
        statements: list[ASTNode] = []
        while not self._check(TokenType.RBRACE) and not self._is_at_end
            ():
             statements.append(self._statement())
        self._expect(TokenType.RBRACE, "Expected '}' to close block")
        return Block(statements=statements, line=tok.line, column=tok.
            column)

    #
        ----------------------------------------------------------------------
```

```
151     # Parameters (function definition)
152     #
            ------------------------------------------------------------------------

153
154     def _params(self) -> list[str]:
155         """IDENTIFIER (',' IDENTIFIER)*  -- returns list of param names.
                """
156         params: list[str] = []
157         if self._check(TokenType.IDENTIFIER):
158             params.append(self._advance().lexeme)
159             while self._check(TokenType.COMMA):
160                 self._advance()                                  # ','
161                 tok = self._expect(TokenType.IDENTIFIER, "Expected
                        parameter name after ','")
162                 params.append(tok.lexeme)
163         return params
164
165     #
            ------------------------------------------------------------------------

166     # Expressions  (comparison is lowest precedence, then additive,
            then term)
167     #
            ------------------------------------------------------------------------

168
169     def _expr(self) -> ASTNode:
170         """additive (('==' | '!=') additive)?  -- non-associative
                comparison"""
171         node = self._additive()
172         if self._check(TokenType.EQUAL_EQUAL) or self._check(TokenType.
                BANG_EQUAL):
173             op_tok = self._advance()
174             right = self._additive()
175             node = BinaryOp(op=op_tok.lexeme, left=node, right=right,
176                             line=op_tok.line, column=op_tok.column)
177         return node
178
179     def _additive(self) -> ASTNode:
180         """term (('+' | '-' | '+.' | '-.') term)*"""
181         node = self._term()
182         while self._check(TokenType.PLUS) or self._check(TokenType.
                MINUS) \
183                 or self._check(TokenType.PLUS_DOT) or self._check(
                    TokenType.MINUS_DOT):
184             op_tok = self._advance()
185             right = self._term()
186             node = BinaryOp(op=op_tok.lexeme, left=node, right=right,
187                             line=op_tok.line, column=op_tok.column)
188         return node
189
190     def _term(self) -> ASTNode:
```

```
191         """factor (('*' | '/' | '*.' | '/.') factor)*"""
192         node = self._factor()
193         while self._check(TokenType.STAR) or self._check(TokenType.
                SLASH) \
194                 or self._check(TokenType.STAR_DOT) or self._check(
                        TokenType.SLASH_DOT):
195             op_tok = self._advance()
196             right = self._factor()
197             node = BinaryOp(op=op_tok.lexeme, left=node, right=right,
198                             line=op_tok.line, column=op_tok.column)
199         return node
200 
201     def _factor(self) -> ASTNode:
202         """
203         '-' factor  (unary negation, desugared to 0 - factor)
204         | INT_LITERAL | FLOAT_LITERAL | STRING_LITERAL | BOOL_LITERAL
205         | IDENTIFIER ( '(' args? ')' )?
206         | '(' expr ')'
207         """
208         tok = self._peek()
209 
210         # Unary minus  '-' factor  (integer negation, desugared to 0 -
                factor)
211         if tok.token_type == TokenType.MINUS:
212             op_tok = self._advance()
213             operand = self._factor()
214             zero = Literal(value=0, line=op_tok.line, column=op_tok.
                        column)
215             return BinaryOp(op="-", left=zero, right=operand,
216                             line=op_tok.line, column=op_tok.column)
217 
218         # Unary minus-dot  '-.' factor  (float negation, desugared to
                0.0 -. factor)
219         if tok.token_type == TokenType.MINUS_DOT:
220             op_tok = self._advance()
221             operand = self._factor()
222             zero_f = Literal(value=0.0, line=op_tok.line, column=op_tok
                        .column)
223             return BinaryOp(op="-.", left=zero_f, right=operand,
224                             line=op_tok.line, column=op_tok.column)
225 
226         # Integer literal
227         if tok.token_type == TokenType.INT_LITERAL:
228             self._advance()
229             return Literal(value=int(tok.lexeme), line=tok.line, column
                        =tok.column)
230 
231         # Float literal
232         if tok.token_type == TokenType.FLOAT_LITERAL:
233             self._advance()
234             return Literal(value=float(tok.lexeme), line=tok.line,
                        column=tok.column)
235 
```

```
        # Boolean literal
        if tok.token_type == TokenType.BOOL_LITERAL:
            self._advance()
            return Literal(value=(tok.lexeme == "true"), line=tok.line,
                column=tok.column)

        # String literal  (keep the surrounding quotes stripped)
        if tok.token_type == TokenType.STRING_LITERAL:
            self._advance()
            return Literal(value=tok.lexeme[1:-1], line=tok.line,
               column=tok.column)

        # Identifier -- could be a plain variable or a function call
        if tok.token_type == TokenType.IDENTIFIER:
            self._advance()
            if self._check(TokenType.LPAREN):
                # function call
                self._advance()                                  # '('
                args = self._args()
                self._expect(TokenType.RPAREN, "Expected ')' after
                   function arguments")
                return FunctionCall(name=tok.lexeme, args=args,
                                    line=tok.line, column=tok.column)
            return Identifier(name=tok.lexeme, line=tok.line, column=
               tok.column)

        # Grouped expression  '(' expr ')'
        if tok.token_type == TokenType.LPAREN:
            self._advance()                                      # '('
            node = self._expr()
            self._expect(TokenType.RPAREN, "Expected ')' to close
               grouped expression")
            return node

        raise ParseError(self._error_at(tok, f"Unexpected token: {tok.
           lexeme!r}"))

    #
       ------------------------------------------------------------------------

    # Arguments (function call)
    #
       ------------------------------------------------------------------------


    def _args(self) -> list[ASTNode]:
        """expr (',' expr)*"""
        args: list[ASTNode] = []
        if not self._check(TokenType.RPAREN):
            args.append(self._expr())
            while self._check(TokenType.COMMA):
                self._advance()                                  # ','
                args.append(self._expr())
```

```
        return args

    # ------------------------------------------------------------------------

    # Token navigation helpers
    # ------------------------------------------------------------------------

    def _peek(self) -> Token:
        return self._tokens[self._pos]

    def _peek_next(self) -> Token:
        if self._pos + 1 < len(self._tokens):
            return self._tokens[self._pos + 1]
        return self._tokens[-1]                                  # EOF

    def _advance(self) -> Token:
        tok = self._tokens[self._pos]
        if not self._is_at_end():
            self._pos += 1
        return tok

    def _check(self, token_type: TokenType) -> bool:
        return self._peek().token_type == token_type

    def _is_at_end(self) -> bool:
        return self._peek().token_type == TokenType.EOF

    def _expect(self, token_type: TokenType, message: str) -> Token:
        if self._check(token_type):
            return self._advance()
        raise ParseError(self._error_at(self._peek(), message))

    def _error_at(self, tok: Token, message: str) -> str:
        return f"[line {tok.line}, col {tok.column}] ParseError: {
            message}"
```

Listing 8.6: components/parser.py — recursive-descent parser

```
from __future__ import annotations

from dataclasses import dataclass, field

from components.ast_nodes import (
    ASTNode,
    Assign,
    BinaryOp,
    Block,
    FunctionCall,
    FunctionDef,
    Identifier,
```

```
    If,
    Literal,
    Program,
    Return,
    While,
)


@dataclass
class ParseTreeNode:
    label: str
    children: list["ParseTreeNode"] = field(default_factory=list)


class ParseTreePrinter:
    """Renders a textbook-style concrete parse tree using ASCII branches."""

    def print(self, node: Program) -> str:
        root = self._program_node(node)
        lines: list[str] = []
        self._render(root, lines, prefix="", is_last=True, is_root=True)
        return "\n".join(lines)

    def _render(
        self,
        node: ParseTreeNode,
        lines: list[str],
        prefix: str,
        is_last: bool,
        is_root: bool = False,
    ) -> None:
        if is_root:
            lines.append(node.label)
        else:
            branch = "`-- " if is_last else "|-- "
            lines.append(f"{prefix}{branch}{node.label}")

        child_prefix = prefix + ("    " if is_last else "|   ")
        for index, child in enumerate(node.children):
            self._render(
                child,
                lines,
                prefix=child_prefix if not is_root else "",
                is_last=index == len(node.children) - 1,
            )

    def _program_node(self, node: Program) -> ParseTreeNode:
        children = [self._statement_node(statement) for statement in node.statements]
        children.append(ParseTreeNode("EOF"))
        return ParseTreeNode("program", children)
```

```
63
64      def _statement_node(self, node: ASTNode) -> ParseTreeNode:
65          if isinstance(node, Assign):
66              return ParseTreeNode("statement", [self._assignment_node(
                    node)])
67          if isinstance(node, If):
68              return ParseTreeNode("statement", [self._if_node(node)])
69          if isinstance(node, While):
70              return ParseTreeNode("statement", [self._while_node(node)])
71          if isinstance(node, FunctionDef):
72              return ParseTreeNode("statement", [self._function_def_node(
                    node)])
73          if isinstance(node, Return):
74              return ParseTreeNode("statement", [self._return_node(node)
                    ])
75          return ParseTreeNode(
76              "statement",
77              [
78                  ParseTreeNode(
79                      "expr_stmt",
80                      [self._expr_node(node), ParseTreeNode("';'")],
81                  )
82              ],
83          )
84
85      def _assignment_node(self, node: Assign) -> ParseTreeNode:
86          return ParseTreeNode(
87              "assignment",
88              [
89                  ParseTreeNode(f"IDENTIFIER({node.name})"),
90                  ParseTreeNode("'='"),
91                  self._expr_node(node.value),
92                  ParseTreeNode("';'"),
93              ],
94          )
95
96      def _if_node(self, node: If) -> ParseTreeNode:
97          children = [
98              ParseTreeNode("'if'"),
99              ParseTreeNode("'('"),
100             self._expr_node(node.condition),
101             ParseTreeNode("')'"),
102             self._block_node(node.then_block),
103         ]
104         if node.else_block is not None:
105             children.extend([ParseTreeNode("'else'"), self._block_node(
                    node.else_block)])
106         return ParseTreeNode("if_stmt", children)
107
108     def _while_node(self, node: While) -> ParseTreeNode:
109         return ParseTreeNode(
110             "while_stmt",
111             [
```

```
                ParseTreeNode("'while'"),
                ParseTreeNode("'('"),
                self._expr_node(node.condition),
                ParseTreeNode("')'"),
                self._block_node(node.body),
            ],
        )

    def _function_def_node(self, node: FunctionDef) -> ParseTreeNode:
        return ParseTreeNode(
            "func_def",
            [
                ParseTreeNode("'def'"),
                ParseTreeNode(f"IDENTIFIER({node.name})"),
                ParseTreeNode("'('"),
                self._params_node(node.params),
                ParseTreeNode("')'"),
                self._block_node(node.body),
            ],
        )

    def _params_node(self, params: list[str]) -> ParseTreeNode:
        if not params:
            return ParseTreeNode("params", [ParseTreeNode("epsilon")])

        children: list[ParseTreeNode] = []
        for index, param in enumerate(params):
            children.append(ParseTreeNode(f"IDENTIFIER({param})"))
            if index < len(params) - 1:
                children.append(ParseTreeNode("','"))
        return ParseTreeNode("params", children)

    def _return_node(self, node: Return) -> ParseTreeNode:
        return ParseTreeNode(
            "return_stmt",
            [
                ParseTreeNode("'return'"),
                self._expr_node(node.expr),
                ParseTreeNode("';'"),
            ],
        )

    def _block_node(self, node: Block) -> ParseTreeNode:
        children = [ParseTreeNode("'{'")]
        children.extend(self._statement_node(statement) for statement
            in node.statements)
        children.append(ParseTreeNode("'}'"))
        return ParseTreeNode("block", children)

    def _expr_node(self, node: ASTNode) -> ParseTreeNode:
        if isinstance(node, BinaryOp) and node.op in {"==", "!="}:
            child = ParseTreeNode(
                "comparison",
```

```
                [
                    self._additive_node(node.left),
                    ParseTreeNode(f"'{node.op}'"),
                    self._additive_node(node.right),
                ],
            )
            return ParseTreeNode("expr", [child])
        return ParseTreeNode("expr", [self._additive_node(node)])

    def _additive_node(self, node: ASTNode) -> ParseTreeNode:
        if isinstance(node, BinaryOp) and node.op in {"+", "-", "+.", "
            -."}:
            return ParseTreeNode(
                "additive",
                [
                    self._additive_node(node.left),
                    ParseTreeNode(f"'{node.op}'"),
                    self._term_node(node.right),
                ],
            )
        return ParseTreeNode("additive", [self._term_node(node)])

    def _term_node(self, node: ASTNode) -> ParseTreeNode:
        if isinstance(node, BinaryOp) and node.op in {"*", "/", "*.", "
            /."}:
            return ParseTreeNode(
                "term",
                [
                    self._term_node(node.left),
                    ParseTreeNode(f"'{node.op}'"),
                    self._factor_node(node.right),
                ],
            )
        return ParseTreeNode("term", [self._factor_node(node)])

    def _factor_node(self, node: ASTNode) -> ParseTreeNode:
        if self._is_integer_unary_minus(node):
            binary = node
            assert isinstance(binary, BinaryOp)
            return ParseTreeNode("factor", [ParseTreeNode("'-'"), self.
                _factor_node(binary.right)])
        if self._is_float_unary_minus(node):
            binary = node
            assert isinstance(binary, BinaryOp)
            return ParseTreeNode("factor", [ParseTreeNode("'-.'"), self
                ._factor_node(binary.right)])
        if isinstance(node, Literal):
            return ParseTreeNode("factor", [ParseTreeNode(self.
                _literal_label(node.value))])
        if isinstance(node, Identifier):
            return ParseTreeNode("factor", [ParseTreeNode(f"IDENTIFIER
                ({node.name})")])
        if isinstance(node, FunctionCall):
```

```
            return ParseTreeNode("factor", [self._function_call_node(
                node)])
        return ParseTreeNode(
            "factor",
            [ParseTreeNode("'('"), self._expr_node(node), ParseTreeNode
                ("')'")],
        )

    def _function_call_node(self, node: FunctionCall) -> ParseTreeNode:
        return ParseTreeNode(
            "function_call",
            [
                ParseTreeNode(f"IDENTIFIER({node.name})"),
                ParseTreeNode("'('"),
                self._args_node(node.args),
                ParseTreeNode("')'"),
            ],
        )

    def _args_node(self, args: list[ASTNode]) -> ParseTreeNode:
        if not args:
            return ParseTreeNode("args", [ParseTreeNode("epsilon")])

        children: list[ParseTreeNode] = []
        for index, arg in enumerate(args):
            children.append(self._expr_node(arg))
            if index < len(args) - 1:
                children.append(ParseTreeNode("','"))
        return ParseTreeNode("args", children)

    @staticmethod
    def _is_integer_unary_minus(node: ASTNode) -> bool:
        return (
            isinstance(node, BinaryOp)
            and node.op == "-"
            and isinstance(node.left, Literal)
            and node.left.value == 0
        )

    @staticmethod
    def _is_float_unary_minus(node: ASTNode) -> bool:
        return (
            isinstance(node, BinaryOp)
            and node.op == "-."
            and isinstance(node.left, Literal)
            and node.left.value == 0.0
        )

    @staticmethod
    def _literal_label(value: object) -> str:
        if isinstance(value, bool):
            return f"BOOL_LITERAL({'true' if value else 'false'})"
        if isinstance(value, int):
```

```
            return f"INT_LITERAL({value})"
        if isinstance(value, float):
            return f"FLOAT_LITERAL({value})"
        if isinstance(value, str):
            return f'STRING_LITERAL("{value}")'
        return f"LITERAL({value!r})"
```

Listing 8.7: components/parse_tree_printer.py — parse-tree renderer

```
from __future__ import annotations

from components.ast_nodes import (
    ASTNode,
    Program,
    Block,
    Assign,
    If,
    While,
    FunctionDef,
    FunctionCall,
    Return,
    BinaryOp,
    Literal,
    Identifier,
)


class ASTPrinter:
    """Walks an AST and returns a human-readable, indented string.

    Usage:
        tree = Parser(tokens).parse()
        print(ASTPrinter().print(tree))

    Output style -- resembles the original source but annotated with
       node types,
    making it easy to verify the tree structure during development and
       in reports.
    """

    INDENT = "    "  # 4 spaces per level

    def print(self, node: ASTNode) -> str:
        lines: list[str] = []
        self._visit(node, lines, level=0)
        return "\n".join(lines)

    #
       ---------------------------------------------------------------------

    # Dispatcher
    #
       ---------------------------------------------------------------------
```

```
40
41      def _visit(self, node: ASTNode, lines: list[str], level: int) ->
            None:
42          method_name = "_visit_" + type(node).__name__
43          visitor = getattr(self, method_name, self._visit_unknown)
44          visitor(node, lines, level)
45
46      def _pad(self, level: int) -> str:
47          return self.INDENT * level
48
49      #
            ------------------------------------------------------------------------
50      # Node visitors
51      #
            ------------------------------------------------------------------------
52
53      def _visit_Program(self, node: Program, lines: list[str], level:
            int) -> None:
54          lines.append(f"{self._pad(level)}Program  [{len(node.statements
                )} statement(s)]")
55          for stmt in node.statements:
56              self._visit(stmt, lines, level + 1)
57
58      def _visit_Block(self, node: Block, lines: list[str], level: int)
            -> None:
59          lines.append(f"{self._pad(level)}Block  [{len(node.statements)}
                statement(s)]")
60          for stmt in node.statements:
61              self._visit(stmt, lines, level + 1)
62
63      def _visit_Assign(self, node: Assign, lines: list[str], level: int)
             -> None:
64          lines.append(f"{self._pad(level)}Assign  {node.name!r}  =")
65          self._visit(node.value, lines, level + 1)
66
67      def _visit_If(self, node: If, lines: list[str], level: int) -> None
            :
68          lines.append(f"{self._pad(level)}If")
69          lines.append(f"{self._pad(level + 1)}condition:")
70          self._visit(node.condition, lines, level + 2)
71          lines.append(f"{self._pad(level + 1)}then:")
72          self._visit(node.then_block, lines, level + 2)
73          if node.else_block is not None:
74              lines.append(f"{self._pad(level + 1)}else:")
75              self._visit(node.else_block, lines, level + 2)
76
77      def _visit_While(self, node: While, lines: list[str], level: int)
            -> None:
78          lines.append(f"{self._pad(level)}While")
79          lines.append(f"{self._pad(level + 1)}condition:")
```

```
        self._visit(node.condition, lines, level + 2)
        lines.append(f"{self._pad(level + 1)}body:")
        self._visit(node.body, lines, level + 2)

    def _visit_FunctionDef(self, node: FunctionDef, lines: list[str], level: int) -> None:
        params = ", ".join(node.params) if node.params else "(none)"
        lines.append(f"{self._pad(level)}FunctionDef  {node.name!r}  params=({params})")
        self._visit(node.body, lines, level + 1)

    def _visit_FunctionCall(self, node: FunctionCall, lines: list[str], level: int) -> None:
        lines.append(f"{self._pad(level)}FunctionCall  {node.name!r}  [{len(node.args)} arg(s)]")
        for i, arg in enumerate(node.args):
            lines.append(f"{self._pad(level + 1)}arg[{i}]:")
            self._visit(arg, lines, level + 2)

    def _visit_Return(self, node: Return, lines: list[str], level: int) -> None:
        lines.append(f"{self._pad(level)}Return")
        self._visit(node.expr, lines, level + 1)

    def _visit_BinaryOp(self, node: BinaryOp, lines: list[str], level: int) -> None:
        lines.append(f"{self._pad(level)}BinaryOp  '{node.op}'")
        lines.append(f"{self._pad(level + 1)}left:")
        self._visit(node.left, lines, level + 2)
        lines.append(f"{self._pad(level + 1)}right:")
        self._visit(node.right, lines, level + 2)

    def _visit_Literal(self, node: Literal, lines: list[str], level: int) -> None:
        type_name = type(node.value).__name__
        lines.append(f"{self._pad(level)}Literal  {node.value!r}  ({type_name})")

    def _visit_Identifier(self, node: Identifier, lines: list[str], level: int) -> None:
        lines.append(f"{self._pad(level)}Identifier  {node.name!r}")

    def _visit_unknown(self, node: ASTNode, lines: list[str], level: int) -> None:
        lines.append(f"{self._pad(level)}??? Unknown node: {type(node).__name__}")
```

Listing 8.8: components/ast_printer.py — AST pretty-printer

```
from __future__ import annotations

from dataclasses import dataclass, field
from enum import Enum
```

```python


class LanguageType(Enum):
    INTEGER = "Integer"
    FLOAT = "Float"
    BOOLEAN = "Boolean"
    STRING = "String"
    VOID = "Void"


@dataclass
class Symbol:
    name: str
    symbol_type: LanguageType


@dataclass
class VariableSymbol(Symbol):
    initialized: bool = False


@dataclass
class FunctionSymbol(Symbol):
    parameters: list[tuple[str, LanguageType]] = field(default_factory=
        list)


class SymbolTableError(Exception):
    pass


class SymbolTable:
    def __init__(self, parent: SymbolTable | None = None) -> None:
        self.parent = parent
        self._symbols: dict[str, Symbol] = {}

    def define_variable(self, name: str, symbol_type: LanguageType,
        initialized: bool = False) -> VariableSymbol:
        if name in self._symbols:
            raise SymbolTableError(f"Symbol already defined in this
                scope: {name}")
        symbol = VariableSymbol(name=name, symbol_type=symbol_type,
            initialized=initialized)
        self._symbols[name] = symbol
        return symbol

    def define_function(
        self,
        name: str,
        return_type: LanguageType,
        parameters: list[tuple[str, LanguageType]],
    ) -> FunctionSymbol:
        if name in self._symbols:
```

```
            raise SymbolTableError(f"Symbol already defined in this scope: {name}")
        symbol = FunctionSymbol(name=name, symbol_type=return_type, parameters=parameters)
        self._symbols[name] = symbol
        return symbol

    def lookup(self, name: str) -> Symbol:
        symbol = self._symbols.get(name)
        if symbol is not None:
            return symbol
        if self.parent is not None:
            return self.parent.lookup(name)
        raise SymbolTableError(f"Undefined symbol: {name}")

    def exists_in_current_scope(self, name: str) -> bool:
        return name in self._symbols

    def child_scope(self) -> SymbolTable:
        return SymbolTable(parent=self)

    def set_initialized(self, name: str) -> None:
        symbol = self.lookup(name)
        if not isinstance(symbol, VariableSymbol):
            raise SymbolTableError(f"Cannot initialize non-variable symbol: {name}")
        symbol.initialized = True

    def snapshot(self) -> dict[str, Symbol]:
        return dict(self._symbols)

    def format_table(self) -> str:
        lines = [
            f"{'Name':<12} {'Kind':<10} {'Type':<10} {'Initialized':<12} Parameters",
            "-" * 72,
        ]

        for name, symbol in self._symbols.items():
            if isinstance(symbol, FunctionSymbol):
                params = ", ".join(f"{param_name}: {param_type.value}"
                    for param_name, param_type in symbol.parameters)
                lines.append(
                    f"{name:<12} {'function':<10} {symbol.symbol_type.value:<10} {'-':<12} {params or '-'}"
                )
            elif isinstance(symbol, VariableSymbol):
                initialized = "yes" if symbol.initialized else "no"
                lines.append(
                    f"{name:<12} {'variable':<10} {symbol.symbol_type.value:<10} {initialized:<12} -"
                )
            else:
```

```
                lines.append(f"{name:<12} {'symbol':<10} {symbol.
                    symbol_type.value:<10} {'-':<12} -")

        return "\n".join(lines)
```

Listing 8.9: components/symbol_table.py — scoped symbol table

```
from __future__ import annotations

from dataclasses import dataclass

from components.ast_nodes import (
    ASTNode,
    Assign,
    BinaryOp,
    Block,
    FunctionCall,
    FunctionDef,
    Identifier,
    If,
    Literal,
    Program,
    Return,
    While,
)
from components.symbol_table import FunctionSymbol, LanguageType,
    SymbolTable, SymbolTableError, VariableSymbol


class TypeCheckError(Exception):
    pass


@dataclass
class _FunctionState:
    node: FunctionDef
    body_checked: bool = False


@dataclass
class _FunctionContext:
    name: str
    return_type: LanguageType | None = None


class TypeChecker:
    _BUILTIN_PRINT = "print"

    def __init__(self) -> None:
        self.global_scope = SymbolTable()
        self._functions: dict[str, _FunctionState] = {}
        self._active_calls: list[str] = []

```

```
    def check(self, program: Program) -> SymbolTable:
        for statement in program.statements:
            if isinstance(statement, FunctionDef):
                self._register_function(statement)

        for statement in program.statements:
            if isinstance(statement, FunctionDef):
                continue
            self._check_statement(statement, self.global_scope, None)

        self._check_uncalled_functions()
        return self.global_scope

    def _register_function(self, node: FunctionDef) -> None:
        if node.name in self._functions:
            raise TypeCheckError(self._error_at(node, f"Function '{node
                .name}' is already defined"))

        placeholder_parameters = [(param_name, LanguageType.VOID) for
            param_name in node.params]
        try:
            self.global_scope.define_function(node.name, LanguageType.
                VOID, placeholder_parameters)
        except SymbolTableError as error:
            raise TypeCheckError(self._error_at(node, str(error))) from
                error
        self._functions[node.name] = _FunctionState(node=node)

    def _check_statement(
        self,
        node: ASTNode,
        scope: SymbolTable,
        function_context: _FunctionContext | None,
    ) -> None:
        if isinstance(node, Assign):
            value_type = self._infer_expr_type(node.value, scope)
            self._assign_variable_type(scope, node, value_type)
            return

        if isinstance(node, Return):
            if function_context is None:
                raise TypeCheckError(self._error_at(node, "'return' is
                    only valid inside a function"))
            expr_type = self._infer_expr_type(node.expr, scope)
            if function_context.return_type is None:
                function_context.return_type = expr_type
            elif function_context.return_type != expr_type:
                raise TypeCheckError(
                    self._error_at(
                        node,
                        f"Function '{function_context.name}' returns
                            both {function_context.return_type.value}
                            and {expr_type.value}",
```

```
                    )
                )
            return

        if isinstance(node, If):
            condition_type = self._infer_expr_type(node.condition,
                scope)
            if condition_type != LanguageType.BOOLEAN:
                raise TypeCheckError(self._error_at(node.condition, "If
                    condition must have type Boolean"))
            self._check_block(node.then_block, scope.child_scope(),
                function_context)
            if node.else_block is not None:
                self._check_block(node.else_block, scope.child_scope(),
                    function_context)
            return

        if isinstance(node, While):
            condition_type = self._infer_expr_type(node.condition,
                scope)
            if condition_type != LanguageType.BOOLEAN:
                raise TypeCheckError(self._error_at(node.condition, "
                    While condition must have type Boolean"))
            self._check_block(node.body, scope.child_scope(),
                function_context)
            return

        if isinstance(node, Block):
            self._check_block(node, scope.child_scope(),
                function_context)
            return

        if isinstance(node, FunctionCall):
            self._infer_function_call_type(node, scope)
            return

        raise TypeCheckError(self._error_at(node, f"Unsupported
            statement node: {type(node).__name__}"))

    def _check_block(
        self,
        block: Block,
        scope: SymbolTable,
        function_context: _FunctionContext | None,
    ) -> None:
        for statement in block.statements:
            self._check_statement(statement, scope, function_context)

    def _infer_expr_type(self, node: ASTNode, scope: SymbolTable) ->
        LanguageType:
        if isinstance(node, Literal):
            return self._literal_type(node)

```

```
135         if isinstance(node, Identifier):
136             try:
137                 symbol = scope.lookup(node.name)
138             except SymbolTableError as error:
139                 raise TypeCheckError(self._error_at(node, str(error)))
                        from error
140             if not isinstance(symbol, VariableSymbol):
141                 raise TypeCheckError(self._error_at(node, f"'{node.name
                        }' is not a variable"))
142             return symbol.symbol_type
143 
144         if isinstance(node, FunctionCall):
145             return self._infer_function_call_type(node, scope)
146 
147         if isinstance(node, BinaryOp):
148             left_type = self._infer_expr_type(node.left, scope)
149             right_type = self._infer_expr_type(node.right, scope)
150             return self._infer_binary_type(node, left_type, right_type)
151 
152         raise TypeCheckError(self._error_at(node, f"Unsupported
                expression node: {type(node).__name__}"))
153 
154     def _infer_function_call_type(self, node: FunctionCall, scope:
            SymbolTable) -> LanguageType:
155         if node.name == self._BUILTIN_PRINT:
156             return self._infer_builtin_print_type(node, scope)
157 
158         function_state = self._functions.get(node.name)
159         if function_state is None:
160             raise TypeCheckError(self._error_at(node, f"Undefined
                    function: {node.name}"))
161         if node.name in self._active_calls:
162             raise TypeCheckError(self._error_at(node, f"Recursive
                    function calls are not supported: {node.name}"))
163 
164         argument_types = [self._infer_expr_type(argument, scope) for
                argument in node.args]
165         function_symbol = self._lookup_function_symbol(node)
166 
167         if len(argument_types) != len(function_symbol.parameters):
168             raise TypeCheckError(
169                 self._error_at(
170                     node,
171                     f"Function '{node.name}' expects {len(
                            function_symbol.parameters)} argument(s), got {
                            len(argument_types)}",
172                 )
173             )
174 
175         declared_parameter_types = [param_type for _, param_type in
                function_symbol.parameters]
176         if all(param_type == LanguageType.VOID for param_type in
                declared_parameter_types):
```

```
177             function_symbol.parameters = list(zip(function_state.node.
                    params, argument_types, strict=False))
178         else:
179             for index, ((_, expected_type), actual_type) in enumerate(
                    zip(function_symbol.parameters, argument_types, strict=
                    False)):
180                 if expected_type != actual_type:
181                     raise TypeCheckError(
182                         self._error_at(
183                             node.args[index],
184                             f"Argument {index + 1} of '{node.name}'
                                    must be {expected_type.value}, got {
                                    actual_type.value}",
185                         )
186                     )
187 
188         if not function_state.body_checked:
189             self._check_function_body(function_state, function_symbol)
190 
191         return function_symbol.symbol_type
192 
193     def _infer_builtin_print_type(self, node: FunctionCall, scope:
            SymbolTable) -> LanguageType:
194         if len(node.args) != 1:
195             raise TypeCheckError(
196                 self._error_at(node, f"Built-in function 'print'
                        expects 1 argument, got {len(node.args)}")
197             )
198         self._infer_expr_type(node.args[0], scope)
199         return LanguageType.VOID
200 
201     def _check_function_body(self, function_state: _FunctionState,
            function_symbol: FunctionSymbol) -> None:
202         function_context = _FunctionContext(name=function_state.node.
                name)
203         function_scope = self.global_scope.child_scope()
204         for parameter_name, parameter_type in function_symbol.
                parameters:
205             function_scope.define_variable(parameter_name,
                    parameter_type, initialized=True)
206 
207         self._active_calls.append(function_state.node.name)
208         try:
209             self._check_block(function_state.node.body, function_scope,
                        function_context)
210         finally:
211             self._active_calls.pop()
212 
213         function_symbol.symbol_type = function_context.return_type or
                LanguageType.VOID
214         function_state.body_checked = True
215 
216     def _check_uncalled_functions(self) -> None:
```

```
        """After all call-site checks, type-check any function whose
           body was never visited."""
        for name, state in self._functions.items():
            if not state.body_checked:
                try:
                    function_symbol = self.global_scope.lookup(name)
                except SymbolTableError:
                    continue
                if not isinstance(function_symbol, FunctionSymbol):
                    continue
                inferred_params = self._infer_param_types_from_body(
                   state.node)
                function_symbol.parameters = inferred_params
                self._check_function_body(state, function_symbol)

    def _infer_param_types_from_body(self, node: FunctionDef) -> list[
       tuple[str, LanguageType]]:
        """Infer parameter types by scanning the body for expression
           contexts.

        Any parameter used as an operand of a float operator (+. -. *.
           /.) is
        inferred as Float; a parameter used with an integer operator (+
            - * /
        == !=) is inferred as Integer.  Parameters with no type hint
           default to
        Integer.
        """
        param_names = set(node.params)
        inferred: dict[str, LanguageType] = {}

        def scan_expr(n: ASTNode) -> None:
            if isinstance(n, BinaryOp):
                if n.op in {"+", "-", "*", "/", "==", "!="}:
                    for operand in (n.left, n.right):
                        if isinstance(operand, Identifier) and operand.
                           name in param_names:
                            inferred.setdefault(operand.name,
                               LanguageType.INTEGER)
                elif n.op in {"+.", "-.", "*.", "/."}:
                    for operand in (n.left, n.right):
                        if isinstance(operand, Identifier) and operand.
                           name in param_names:
                            inferred.setdefault(operand.name,
                               LanguageType.FLOAT)
                scan_expr(n.left)
                scan_expr(n.right)
            elif isinstance(n, FunctionCall):
                for arg in n.args:
                    scan_expr(arg)

        def scan_stmt(n: ASTNode) -> None:
            if isinstance(n, Assign):
```

```
                scan_expr(n.value)
            elif isinstance(n, Return):
                scan_expr(n.expr)
            elif isinstance(n, If):
                scan_expr(n.condition)
                scan_block(n.then_block)
                if n.else_block:
                    scan_block(n.else_block)
            elif isinstance(n, While):
                scan_expr(n.condition)
                scan_block(n.body)
            elif isinstance(n, Block):
                scan_block(n)
            elif isinstance(n, FunctionCall):
                for arg in n.args:
                    scan_expr(arg)

        def scan_block(block: Block) -> None:
            for stmt in block.statements:
                scan_stmt(stmt)

        scan_block(node.body)
        return [(param, inferred.get(param, LanguageType.INTEGER)) for
           param in node.params]

    def _assign_variable_type(self, scope: SymbolTable, node: Assign,
       value_type: LanguageType) -> None:
        if value_type == LanguageType.VOID:
            raise TypeCheckError(self._error_at(node, f"Cannot assign
               the result of a void function to '{node.name}'"))
        try:
            symbol = scope.lookup(node.name)
        except SymbolTableError:
            scope.define_variable(node.name, value_type, initialized=
               True)
            return

        if not isinstance(symbol, VariableSymbol):
            raise TypeCheckError(self._error_at(node, f"'{node.name}'
               is not a variable"))
        if symbol.symbol_type != value_type:
            raise TypeCheckError(
                self._error_at(
                    node,
                    f"Cannot assign {value_type.value} to variable '{
                       node.name}' of type {symbol.symbol_type.value}",
                )
            )
        symbol.initialized = True

    def _infer_binary_type(
        self,
        node: BinaryOp,
```

```
306             left_type: LanguageType,
307             right_type: LanguageType,
308         ) -> LanguageType:
309             # Integer operators: both operands must be Integer
310             if node.op in {"+", "-", "*"}:
311                 if left_type != LanguageType.INTEGER or right_type !=
                        LanguageType.INTEGER:
312                     raise TypeCheckError(
313                         self._error_at(node, f"Operator '{node.op}'
                                requires two Integer operands")
314                     )
315                 return LanguageType.INTEGER
316 
317             # Integer division: both operands must be Integer, result is
                    Integer
318             if node.op == "/":
319                 if left_type != LanguageType.INTEGER or right_type !=
                        LanguageType.INTEGER:
320                     raise TypeCheckError(
321                         self._error_at(node, f"Operator '{node.op}'
                                requires two Integer operands")
322                     )
323                 return LanguageType.INTEGER
324 
325             # Float operators: both operands must be Float
326             if node.op in {"+.", "-.", "*.", "/."}:
327                 if left_type != LanguageType.FLOAT or right_type !=
                        LanguageType.FLOAT:
328                     raise TypeCheckError(
329                         self._error_at(node, f"Operator '{node.op}'
                                requires two Float operands")
330                     )
331                 return LanguageType.FLOAT
332 
333             if node.op in {"==", "!="}:
334                 if not self._is_numeric(left_type) or not self._is_numeric(
                        right_type):
335                     raise TypeCheckError(
336                         self._error_at(
337                             node,
338                             f"Operator '{node.op}' requires two arithmetic
                                    operands",
339                         )
340                     )
341                 if left_type != right_type:
342                     raise TypeCheckError(
343                         self._error_at(
344                             node,
345                             f"Operator '{node.op}' requires operands of the
                                     same type, got {left_type.value} and {
                                    right_type.value}",
346                         )
347                     )
```

```
            return LanguageType.BOOLEAN

        raise TypeCheckError(self._error_at(node, f"Unsupported operator: {node.op}"))

    def _lookup_function_symbol(self, node: FunctionCall) -> FunctionSymbol:
        try:
            symbol = self.global_scope.lookup(node.name)
        except SymbolTableError as error:
            raise TypeCheckError(self._error_at(node, str(error))) from error
        if not isinstance(symbol, FunctionSymbol):
            raise TypeCheckError(self._error_at(node, f"'{node.name}' is not a function"))
        return symbol

    @staticmethod
    def _is_numeric(language_type: LanguageType) -> bool:
        return language_type in {LanguageType.INTEGER, LanguageType.FLOAT}

    @staticmethod
    def _literal_type(node: Literal) -> LanguageType:
        if isinstance(node.value, bool):
            return LanguageType.BOOLEAN
        if isinstance(node.value, int):
            return LanguageType.INTEGER
        if isinstance(node.value, float):
            return LanguageType.FLOAT
        if isinstance(node.value, str):
            return LanguageType.STRING
        raise TypeCheckError(f"Unsupported literal value: {node.value!r}")

    @staticmethod
    def _error_at(node: ASTNode, message: str) -> str:
        return f"[line {node.line}, col {node.column}] TypeError: {message}"
```

Listing 8.10: components/type_checker.py — static type checker with inference

```
from __future__ import annotations

from dataclasses import dataclass
from typing import Callable

from components.ast_nodes import (
    ASTNode,
    Assign,
    BinaryOp,
    Block,
    FunctionCall,
```

```
    FunctionDef,
    Identifier,
    If,
    Literal,
    Program,
    Return,
    While,
)
from components.symbol_table import LanguageType


class InterpreterError(Exception):
    pass


class _ReturnSignal(Exception):
    def __init__(self, value: object) -> None:
        self.value = value


@dataclass
class _FunctionRuntime:
    node: FunctionDef


class _Environment:
    def __init__(self, parent: _Environment | None = None) -> None:
        self.parent = parent
        self.values: dict[str, object] = {}

    def define(self, name: str, value: object) -> None:
        self.values[name] = value

    def get(self, name: str) -> object:
        if name in self.values:
            return self.values[name]
        if self.parent is not None:
            return self.parent.get(name)
        raise InterpreterError(f"Undefined variable: {name}")

    def assign(self, name: str, value: object) -> None:
        if name in self.values:
            self.values[name] = value
            return
        if self.parent is not None and self.parent.contains(name):
            self.parent.assign(name, value)
            return
        self.values[name] = value

    def contains(self, name: str) -> bool:
        if name in self.values:
            return True
        if self.parent is not None:
```

```
            return self.parent.contains(name)
        return False


class Interpreter:
    _BUILTIN_PRINT = "print"

    def __init__(self, output_callback: Callable[[str], None] | None =
        None) -> None:
        self._functions: dict[str, _FunctionRuntime] = {}
        self._output_callback = output_callback or print

    def execute(self, program: Program) -> None:
        environment = _Environment()
        for statement in program.statements:
            if isinstance(statement, FunctionDef):
                self._functions[statement.name] = _FunctionRuntime(node
                    =statement)

        for statement in program.statements:
            if isinstance(statement, FunctionDef):
                continue
            self._execute_statement(statement, environment)

    def _execute_statement(self, node: ASTNode, environment:
        _Environment) -> None:
        if isinstance(node, Assign):
            value = self._evaluate(node.value, environment)
            environment.assign(node.name, value)
            return

        if isinstance(node, Return):
            raise _ReturnSignal(self._evaluate(node.expr, environment))

        if isinstance(node, If):
            condition = self._evaluate(node.condition, environment)
            if not isinstance(condition, bool):
                raise InterpreterError(self._error_at(node.condition, "
                    If condition must evaluate to Boolean"))
            branch = node.then_block if condition else node.else_block
            if branch is not None:
                self._execute_block(branch, _Environment(parent=
                    environment))
            return

        if isinstance(node, While):
            while True:
                condition = self._evaluate(node.condition, environment)
                if not isinstance(condition, bool):
                    raise InterpreterError(self._error_at(node.
                        condition, "While condition must evaluate to
                        Boolean"))
                if not condition:
```

```
                    break
                self._execute_block(node.body, _Environment(parent=
                    environment))
            return

        if isinstance(node, Block):
            self._execute_block(node, _Environment(parent=environment))
            return

        if isinstance(node, FunctionCall):
            self._evaluate(node, environment)
            return

        raise InterpreterError(self._error_at(node, f"Unsupported
            statement node: {type(node).__name__}"))

    def _execute_block(self, block: Block, environment: _Environment)
        -> None:
        for statement in block.statements:
            self._execute_statement(statement, environment)

    def _evaluate(self, node: ASTNode, environment: _Environment) ->
        object:
        if isinstance(node, Literal):
            return node.value

        if isinstance(node, Identifier):
            return environment.get(node.name)

        if isinstance(node, BinaryOp):
            left_value = self._evaluate(node.left, environment)
            right_value = self._evaluate(node.right, environment)
            return self._evaluate_binary(node, left_value, right_value)

        if isinstance(node, FunctionCall):
            return self._call_function(node, environment)

        raise InterpreterError(self._error_at(node, f"Unsupported
            expression node: {type(node).__name__}"))

    def _call_function(self, node: FunctionCall, environment:
        _Environment) -> object:
        if node.name == self._BUILTIN_PRINT:
            return self._call_builtin_print(node, environment)

        function_runtime = self._functions.get(node.name)
        if function_runtime is None:
            raise InterpreterError(self._error_at(node, f"Undefined
                function: {node.name}"))

        if len(node.args) != len(function_runtime.node.params):
            raise InterpreterError(
                self._error_at(
```

```
                    node,
                    f"Function '{node.name}' expects {len(
                        function_runtime.node.params)} argument(s), got
                        {len(node.args)}",
                )
            )

        call_environment = _Environment(parent=environment)
        argument_values = [self._evaluate(argument, environment) for
            argument in node.args]
        for parameter_name, argument_value in zip(function_runtime.node
            .params, argument_values, strict=False):
            call_environment.define(parameter_name, argument_value)

        try:
            self._execute_block(function_runtime.node.body,
                call_environment)
        except _ReturnSignal as signal:
            return signal.value
        return None

    def _call_builtin_print(self, node: FunctionCall, environment:
        _Environment) -> None:
        if len(node.args) != 1:
            raise InterpreterError(
                self._error_at(node, f"Built-in function 'print'
                    expects 1 argument, got {len(node.args)}")
            )
        value = self._evaluate(node.args[0], environment)
        self._output_callback(f"{self._format_value(value)} : {self.
            _runtime_type(value).value}")
        return None

    def _evaluate_binary(self, node: BinaryOp, left_value: object,
        right_value: object) -> object:
        if node.op == "+":
            return left_value + right_value
        if node.op == "-":
            return left_value - right_value
        if node.op == "*":
            return left_value * right_value
        if node.op == "/":
            if int(right_value) == 0:
                raise InterpreterError(self._error_at(node, "Division
                    by zero"))
            return int(left_value) // int(right_value)
        if node.op == "+.":
            return float(left_value) + float(right_value)
        if node.op == "-.":
            return float(left_value) - float(right_value)
        if node.op == "*.":
            return float(left_value) * float(right_value)
        if node.op == "/.":
```

```
            if float(right_value) == 0.0:
                raise InterpreterError(self._error_at(node, "Division
                    by zero"))
            return float(left_value) / float(right_value)
        if node.op == "==":
            return left_value == right_value
        if node.op == "!=":
            return left_value != right_value
        raise InterpreterError(self._error_at(node, f"Unsupported
            operator: {node.op}"))

    @staticmethod
    def _format_value(value: object) -> str:
        if isinstance(value, str):
            return f'"{value}"'
        if isinstance(value, bool):
            return "true" if value else "false"
        return str(value)

    @staticmethod
    def _runtime_type(value: object) -> LanguageType:
        if isinstance(value, bool):
            return LanguageType.BOOLEAN
        if isinstance(value, int):
            return LanguageType.INTEGER
        if isinstance(value, float):
            return LanguageType.FLOAT
        if isinstance(value, str):
            return LanguageType.STRING
        return LanguageType.VOID

    @staticmethod
    def _error_at(node: ASTNode, message: str) -> str:
        return f"[line {node.line}, col {node.column}] RuntimeError: {
            message}"
```

Listing 8.11: components/interpreter.py — tree-walking interpreter

```
from __future__ import annotations

from dataclasses import dataclass

from components.ast_printer import ASTPrinter
from components.interpreter import Interpreter
from components.lexica import Lexer
from components.parse_tree_printer import ParseTreePrinter
from components.parser import Parser
from components.symbol_table import SymbolTable
from components.tokens import Token
from components.type_checker import TypeChecker


@dataclass
```

```
class PipelineResult:
    tokens: list[Token]
    parse_tree_text: str
    ast_text: str
    checked_scope: SymbolTable
    outputs: list[str]


def run_pipeline(source_code: str) -> PipelineResult:
    tokens = Lexer(source_code).tokenize()
    tree = Parser(tokens).parse()
    parse_tree_text = ParseTreePrinter().print(tree)
    ast_text = ASTPrinter().print(tree)
    checked_scope = TypeChecker().check(tree)

    outputs: list[str] = []
    Interpreter(output_callback=outputs.append).execute(tree)

    return PipelineResult(
        tokens=tokens,
        parse_tree_text=parse_tree_text,
        ast_text=ast_text,
        checked_scope=checked_scope,
        outputs=outputs,
    )


def format_tokens(tokens: list[Token]) -> str:
    lines = []
    for token in tokens:
        lines.append(
            f"  {token.token_type.name:<14} {token.lexeme!r:<12} line={
                token.line} col={token.column}"
        )
    return "\n".join(lines)


def format_stage_output(result: PipelineResult) -> str:
    sections = [
        "=" * 60,
        "  STAGE 1 -- TOKENS",
        "=" * 60,
        format_tokens(result.tokens),
        "",
        "=" * 60,
        "  STAGE 2 -- PARSE TREE",
        "=" * 60,
        result.parse_tree_text,
        "",
        "=" * 60,
        "  STAGE 3 -- AST",
        "=" * 60,
        result.ast_text,
```

```
        "",
        "=" * 60,
        "  STAGE 4 -- TYPE CHECK",
        "=" * 60,
        result.checked_scope.format_table(),
        "",
        "=" * 60,
        "  STAGE 5 -- EXECUTION",
        "=" * 60,
        "\n".join(result.outputs) if result.outputs else "(no output)",
    ]
    return "\n".join(sections)
```

Listing 8.12: components/pipeline.py — shared pipeline (CLI, UI, and tests)

A.3 Test Suite

```
from __future__ import annotations

import unittest

from components.interpreter import InterpreterError
from components.lexica import Lexer, LexerError
from components.parser import Parser, ParseError
from components.pipeline import run_pipeline
from components.symbol_table import LanguageType, SymbolTable,
    SymbolTableError
from components.tokens import TokenType
from components.type_checker import TypeCheckError


#
    ------------------------------------------------------------------------

# Lexer
#
    ------------------------------------------------------------------------


class LexerTests(unittest.TestCase):
    """Tests for components/lexica.py -- lexical analysis and
       tokenisation."""

    def test_integer_literal_token(self) -> None:
        tokens = Lexer("42").tokenize()
        self.assertEqual(tokens[0].token_type, TokenType.INT_LITERAL)
        self.assertEqual(tokens[0].lexeme, "42")

    def test_float_literal_token(self) -> None:
        tokens = Lexer("3.14").tokenize()
        self.assertEqual(tokens[0].token_type, TokenType.FLOAT_LITERAL)
        self.assertEqual(tokens[0].lexeme, "3.14")
```

```
30
31      def test_bool_literals_produce_bool_literal_tokens(self) -> None:
32          tokens = Lexer("true false").tokenize()
33          self.assertEqual(tokens[0].token_type, TokenType.BOOL_LITERAL)
34          self.assertEqual(tokens[0].lexeme, "true")
35          self.assertEqual(tokens[1].token_type, TokenType.BOOL_LITERAL)
36          self.assertEqual(tokens[1].lexeme, "false")
37
38      def test_keywords_produce_correct_token_types(self) -> None:
39          tokens = Lexer("if else while def return").tokenize()
40          expected = [
41              TokenType.IF, TokenType.ELSE, TokenType.WHILE, TokenType.
                    DEF, TokenType.RETURN,
42          ]
43          actual = [t.token_type for t in tokens if t.token_type !=
                TokenType.EOF]
44          self.assertEqual(actual, expected)
45
46      def test_identifier_not_confused_with_keyword(self) -> None:
47          tokens = Lexer("iffy whileloop print").tokenize()
48          self.assertEqual(tokens[0].token_type, TokenType.IDENTIFIER)
49          self.assertEqual(tokens[1].token_type, TokenType.IDENTIFIER)
50          self.assertEqual(tokens[2].token_type, TokenType.IDENTIFIER)
51
52      def test_two_character_operators(self) -> None:
53          tokens = Lexer("== !=").tokenize()
54          self.assertEqual(tokens[0].token_type, TokenType.EQUAL_EQUAL)
55          self.assertEqual(tokens[1].token_type, TokenType.BANG_EQUAL)
56
57      def test_float_dot_operators_tokenised(self) -> None:
58          tokens = Lexer("+. -. *. /.").tokenize()
59          expected = [TokenType.PLUS_DOT, TokenType.MINUS_DOT, TokenType.
                STAR_DOT, TokenType.SLASH_DOT]
60          actual = [t.token_type for t in tokens if t.token_type !=
                TokenType.EOF]
61          self.assertEqual(actual, expected)
62
63      def test_float_literal_not_confused_with_dot_operator(self) -> None
            :
64          # "3.14" must produce FLOAT_LITERAL, not INT_LITERAL followed
                by something
65          tokens = Lexer("3.14").tokenize()
66          self.assertEqual(tokens[0].token_type, TokenType.FLOAT_LITERAL)
67          self.assertEqual(tokens[0].lexeme, "3.14")
68
69      def test_source_position_tracked_across_lines(self) -> None:
70          tokens = Lexer("x\ny").tokenize()
71          self.assertEqual(tokens[0].line, 1)
72          self.assertEqual(tokens[1].line, 2)
73
74      def test_unexpected_character_raises_lexer_error(self) -> None:
75          with self.assertRaises(LexerError):
76              Lexer("@").tokenize()
```

```
77
78      def test_lone_exclamation_raises_lexer_error(self) -> None:
79          with self.assertRaises(LexerError):
80              Lexer("!5").tokenize()
81
82      def test_unterminated_string_raises_lexer_error(self) -> None:
83          with self.assertRaises(LexerError):
84              Lexer('"hello').tokenize()
85
86
87  #
        ----------------------------------------------------------------------------

88  # Symbol table
89  #
        ----------------------------------------------------------------------------

90
91  class SymbolTableTests(unittest.TestCase):
92      """Tests for components/symbol_table.py -- scoped symbol table and
            semantic types."""
93
94      def test_define_variable_and_lookup(self) -> None:
95          table = SymbolTable()
96          table.define_variable("x", LanguageType.INTEGER, initialized=
                True)
97          symbol = table.lookup("x")
98          self.assertEqual(symbol.symbol_type, LanguageType.INTEGER)
99
100     def test_duplicate_variable_definition_raises_error(self) -> None:
101         table = SymbolTable()
102         table.define_variable("x", LanguageType.INTEGER)
103         with self.assertRaises(SymbolTableError):
104             table.define_variable("x", LanguageType.FLOAT)
105
106     def test_duplicate_function_definition_raises_error(self) -> None:
107         table = SymbolTable()
108         table.define_function("f", LanguageType.INTEGER, [])
109         with self.assertRaises(SymbolTableError):
110             table.define_function("f", LanguageType.FLOAT, [])
111
112     def test_lookup_in_parent_scope(self) -> None:
113         parent = SymbolTable()
114         parent.define_variable("x", LanguageType.INTEGER, initialized=
                True)
115         child = parent.child_scope()
116         symbol = child.lookup("x")
117         self.assertEqual(symbol.symbol_type, LanguageType.INTEGER)
118
119     def test_undefined_symbol_raises_error(self) -> None:
120         table = SymbolTable()
121         with self.assertRaises(SymbolTableError):
122             table.lookup("z")
```

```
123
124     def test_format_table_contains_variable_row(self) -> None:
125         table = SymbolTable()
126         table.define_variable("counter", LanguageType.INTEGER,
                initialized=True)
127         output = table.format_table()
128         self.assertIn("counter", output)
129         self.assertIn("variable", output)
130         self.assertIn("Integer", output)
131
132     def test_format_table_contains_function_row(self) -> None:
133         table = SymbolTable()
134         table.define_function("add", LanguageType.INTEGER, [("a",
                LanguageType.INTEGER)])
135         output = table.format_table()
136         self.assertIn("add", output)
137         self.assertIn("function", output)
138
139
140 #
        ------------------------------------------------------------------------

141 # Parser
142 #
        ------------------------------------------------------------------------

143
144 class ParserTests(unittest.TestCase):
145     """Tests for components/parser.py -- recursive-descent parsing."""
146
147     def _parse(self, source: str):
148         return Parser(Lexer(source).tokenize()).parse()
149
150     def test_missing_semicolon_raises_parse_error(self) -> None:
151         with self.assertRaises(ParseError):
152             self._parse("x = 5")
153
154     def test_unclosed_brace_raises_parse_error(self) -> None:
155         with self.assertRaises(ParseError):
156             self._parse("if (true) { x = 1;")
157
158     def test_operator_precedence_mul_before_add(self) -> None:
159         # 2 + 3 * 4 must be 14, not 20
160         result = run_pipeline("x = 2 + 3 * 4; print(x);")
161         self.assertEqual(result.outputs, ["14 : Integer"])
162
163     def test_operator_left_associativity(self) -> None:
164         # 10 - 3 - 2 must be 5 (left-assoc), not 9
165         result = run_pipeline("x = 10 - 3 - 2; print(x);")
166         self.assertEqual(result.outputs, ["5 : Integer"])
167
168     def test_parentheses_override_precedence(self) -> None:
169         # (2 + 3) * 4 must be 20
```

```
        result = run_pipeline("x = (2 + 3) * 4; print(x);")
        self.assertEqual(result.outputs, ["20 : Integer"])

    def test_float_operator_precedence_mul_before_add(self) -> None:
        # 1.0 +. 2.0 *. 3.0 must be 7.0 (*.  binds tighter than +.)
        result = run_pipeline("x = 1.0 +. 2.0 *. 3.0; print(x);")
        self.assertEqual(result.outputs, ["7.0 : Float"])

    def test_pipeline_exposes_parse_tree_and_ast(self) -> None:
        result = run_pipeline("x = 5; print(x);")
        self.assertIn("program", result.parse_tree_text)
        self.assertIn("assignment", result.parse_tree_text)
        self.assertIn("additive", result.parse_tree_text)
        self.assertIn("term", result.parse_tree_text)
        self.assertIn("factor", result.parse_tree_text)
        self.assertIn("function_call", result.parse_tree_text)
        self.assertIn("Program", result.ast_text)
        self.assertIn("Assign", result.ast_text)
        self.assertIn("FunctionCall  'print'", result.ast_text)


# ---------------------------------------------------------------------------
# Type checker
# ---------------------------------------------------------------------------


class TypeCheckerTests(unittest.TestCase):
    """Tests for components/type_checker.py -- static type inference and
        checking."""

    def test_integer_arithmetic_stays_integer(self) -> None:
        result = run_pipeline("x = 3 + 4; print(x);")
        self.assertEqual(result.outputs, ["7 : Integer"])

    def test_division_always_produces_integer(self) -> None:
        # integer / integer -> Integer (no implicit float conversion)
        result = run_pipeline("x = 10 / 2; print(x);")
        self.assertEqual(result.outputs, ["5 : Integer"])

    def test_float_dot_operators_produce_float(self) -> None:
        result = run_pipeline("x = 1.0 +. 2.5; print(x);")
        self.assertEqual(result.outputs, ["3.5 : Float"])

    def test_float_subtraction_produces_float(self) -> None:
        result = run_pipeline("x = 5.0 -. 1.5; print(x);")
        self.assertEqual(result.outputs, ["3.5 : Float"])

    def test_float_multiplication_produces_float(self) -> None:
        result = run_pipeline("x = 2.0 *. 3.0; print(x);")
        self.assertEqual(result.outputs, ["6.0 : Float"])
```

```
218
219     def test_float_division_produces_float(self) -> None:
220         result = run_pipeline("x = 9.0 /. 4.0; print(x);")
221         self.assertEqual(result.outputs, ["2.25 : Float"])
222
223     def test_integer_with_float_op_raises_type_error(self) -> None:
224         # +. requires Float operands; integers must not be accepted
225         with self.assertRaises(TypeCheckError):
226             run_pipeline("x = 2 +. 3; print(x);")
227
228     def test_mixed_int_float_addition_raises_type_error(self) -> None:
229         # Integer + Float is no longer valid; must use +.
230         with self.assertRaises(TypeCheckError):
231             run_pipeline("x = 1 + 2.5; print(x);")
232
233     def test_string_variable_type_inferred(self) -> None:
234         result = run_pipeline('x = "hello"; print(x);')
235         self.assertEqual(result.outputs, ['"hello" : String'])
236
237     def test_boolean_literal_type_inferred(self) -> None:
238         result = run_pipeline("x = true; print(x);")
239         self.assertEqual(result.outputs, ["true : Boolean"])
240
241     def test_boolean_equality_raises_type_error(self) -> None:
242         # Boolean == Boolean is not allowed; == only accepts arithmetic
                operands
243         with self.assertRaises(TypeCheckError):
244             run_pipeline('x = true; if (x == false) { print("then"); }
                    else { print("else"); }')
245
246     def test_mixed_int_float_equality_raises_type_error(self) -> None:
247         # == and != must not accept mixed Integer/Float (no overloading
                )
248         with self.assertRaises(TypeCheckError):
249             run_pipeline("x = 5; y = 3.14; if (x == y) { print(x); }")
250
251     def test_mixed_int_float_inequality_raises_type_error(self) -> None
            :
252         with self.assertRaises(TypeCheckError):
253             run_pipeline("x = 5; y = 3.14; if (x != y) { print(x); }")
254
255     def test_arithmetic_on_string_raises_type_error(self) -> None:
256         with self.assertRaises(TypeCheckError):
257             run_pipeline('x = "a" + 1;')
258
259     def test_if_non_boolean_condition_raises_type_error(self) -> None:
260         with self.assertRaises(TypeCheckError):
261             run_pipeline("if (1) { x = 2; }")
262
263     def test_while_non_boolean_condition_raises_type_error(self) ->
            None:
264         with self.assertRaises(TypeCheckError):
265             run_pipeline("x = 0; while (x) { x = x + 1; }")
```

```
266
267     def test_assignment_type_mismatch_raises_type_error(self) -> None:
268         with self.assertRaises(TypeCheckError):
269             run_pipeline('x = 5; x = "hello";')
270
271     def test_undefined_variable_raises_type_error(self) -> None:
272         with self.assertRaises(TypeCheckError):
273             run_pipeline("print(z);")
274
275     def test_function_wrong_arg_count_raises_type_error(self) -> None:
276         with self.assertRaises(TypeCheckError):
277             run_pipeline("def f(a) { return a; } f(1, 2);")
278
279     def test_function_arg_type_mismatch_raises_type_error(self) -> None
        :
280         # first call infers a: Integer; second call with String must
            fail
281         with self.assertRaises(TypeCheckError):
282             run_pipeline('def f(a) { return a; } f(1); f("hello");')
283
284     def test_uncalled_function_body_type_error_is_caught(self) -> None:
285         # type error inside body must be detected even if function is
            never called
286         with self.assertRaises(TypeCheckError):
287             run_pipeline('def bad() { y = "hello" + 1; }')
288
289     def test_valid_uncalled_function_does_not_raise(self) -> None:
290         # a well-typed but uncalled function should not raise
291         result = run_pipeline("def add(a, b) { return a + b; }")
292         self.assertEqual(result.outputs, [])
293
294
295 #
    ------------------------------------------------------------------------

296 # Integration (full pipeline)
297 #
    ------------------------------------------------------------------------

298
299 class IntegrationTests(unittest.TestCase):
300     """End-to-end tests through all four pipeline stages."""
301
302     # --- Unary minus ---
303
304     def test_unary_minus_integer(self) -> None:
305         result = run_pipeline("x = -5; print(x);")
306         self.assertEqual(result.outputs, ["-5 : Integer"])
307
308     def test_unary_minus_float(self) -> None:
309         result = run_pipeline("y = -. 3.14; print(y);")
310         self.assertEqual(result.outputs, ["-3.14 : Float"])
311
```

```
312     def test_unary_minus_dot_variable(self) -> None:
313         result = run_pipeline("x = 1.5; y = -. x; print(y);")
314         self.assertEqual(result.outputs, ["-1.5 : Float"])
315 
316     def test_unary_minus_variable(self) -> None:
317         result = run_pipeline("x = 10; y = -x; print(y);")
318         self.assertEqual(result.outputs, ["-10 : Integer"])
319 
320     def test_unary_minus_inside_expression(self) -> None:
321         result = run_pipeline("x = 3 + -2; print(x);")
322         self.assertEqual(result.outputs, ["1 : Integer"])
323 
324     # --- Control flow ---
325 
326     def test_if_then_branch_executes(self) -> None:
327         result = run_pipeline(
328             'x = 5; if (x == 5) { print("yes"); } else { print("no"); }
                ',
329         )
330         self.assertEqual(result.outputs, ['"yes" : String'])
331 
332     def test_if_else_branch_executes_when_condition_false(self) -> None
        :
333         result = run_pipeline(
334             'x = 0; if (x == 5) { print("yes"); } else { print("no"); }
                ',
335         )
336         self.assertEqual(result.outputs, ['"no" : String'])
337 
338     def test_while_loop_executes_repeatedly(self) -> None:
339         result = run_pipeline("x = 3; while (x != 0) { print(x); x = x
            - 1; }")
340         self.assertEqual(result.outputs, ["3 : Integer", "2 : Integer",
                "1 : Integer"])
341 
342     # --- Division by zero ---
343 
344     def test_integer_division_by_zero_raises_error(self) -> None:
345         with self.assertRaises(InterpreterError):
346             run_pipeline("x = 10 / 0; print(x);")
347 
348     def test_float_division_by_zero_raises_error(self) -> None:
349         with self.assertRaises(InterpreterError):
350             run_pipeline("x = 10.0 /. 0.0; print(x);")
351 
352     # --- Functions ---
353 
354     def test_function_integer_return(self) -> None:
355         result = run_pipeline("def square(n) { return n * n; } print(
            square(7));")
356         self.assertEqual(result.outputs, ["49 : Integer"])
357 
358     def test_function_type_inference(self) -> None:
```

```
359         result = run_pipeline(
360             "def add(a, b) { return a +. b; } result = add(2.0, 3.5);
                print(result);"
361         )
362         self.assertEqual(result.outputs, ["5.5 : Float"])
363         self.assertIn("result       variable   Float", result.
                checked_scope.format_table())
364 
365     def test_function_value_parameter_passing(self) -> None:
366         # Modifying a parameter inside a function must not affect the
                caller's variable
367         result = run_pipeline(
368             "def inc(a) { a = a + 1; return a; } x = 10; y = inc(x);
                print(x); print(y);"
369         )
370         self.assertEqual(result.outputs, ["10 : Integer", "11 : Integer
                "])
371 
372     def test_nested_function_calls(self) -> None:
373         result = run_pipeline(
374             "def add(a, b) { return a + b; } print(add(add(1, 2), 3));"
375         )
376         self.assertEqual(result.outputs, ["6 : Integer"])
377 
378     # --- print() with all four types ---
379 
380     def test_print_integer(self) -> None:
381         result = run_pipeline("print(42);")
382         self.assertEqual(result.outputs, ["42 : Integer"])
383 
384     def test_print_float(self) -> None:
385         result = run_pipeline("print(3.14);")
386         self.assertEqual(result.outputs, ["3.14 : Float"])
387 
388     def test_print_boolean(self) -> None:
389         result = run_pipeline("print(true);")
390         self.assertEqual(result.outputs, ["true : Boolean"])
391 
392     def test_print_string(self) -> None:
393         result = run_pipeline('print("plc");')
394         self.assertEqual(result.outputs, ['"plc" : String'])
395 
396     # --- Boolean expressions as values ---
397 
398     def test_comparison_result_assigned_to_variable(self) -> None:
399         result = run_pipeline("x = 5; y = 5; flag = (x == y); print(
                flag);")
400         self.assertEqual(result.outputs, ["true : Boolean"])
401 
402     def test_comparison_result_printed_directly(self) -> None:
403         result = run_pipeline("x = 10; print(x != 0);")
404         self.assertEqual(result.outputs, ["true : Boolean"])
405 
```

```
406     def test_comparison_result_false(self) -> None:
407         result = run_pipeline("a = 1; b = 2; eq = (a == b); print(eq);"
                )
408         self.assertEqual(result.outputs, ["false : Boolean"])
409 
410     # --- Reassignment ---
411 
412     def test_variable_can_be_reassigned_same_type(self) -> None:
413         result = run_pipeline("x = 1; x = 2; x = 3; print(x);")
414         self.assertEqual(result.outputs, ["3 : Integer"])
415 
416     def test_scope_isolation_if_block(self) -> None:
417         # Variable defined inside an if-block must not be visible
                outside it
418         with self.assertRaises(TypeCheckError):
419             run_pipeline(
420                 "x = 1; if (x == 1) { inner = 99; } print(inner);"
421             )
422 
423     def test_string_equality_raises_type_error(self) -> None:
424         # == is only valid between arithmetic (Integer/Float) operands
425         with self.assertRaises(TypeCheckError):
426             run_pipeline('a = "hello"; b = "world"; if (a == b) { print
                    (a); }')
427 
428     def test_integer_floor_division(self) -> None:
429         # 7 / 2 must be 3 (floor), not 3.5
430         result = run_pipeline("x = 7 / 2; print(x);")
431         self.assertEqual(result.outputs, ["3 : Integer"])
432 
433     def test_void_function_assignment_raises_type_error(self) -> None:
434         # Assigning the result of a void function (no return) must be a
                type error
435         with self.assertRaises(TypeCheckError):
436             run_pipeline("def greet() { print(42); } x = greet();")
437 
438     def test_builtin_print_assignment_raises_type_error(self) -> None:
439         with self.assertRaises(TypeCheckError):
440             run_pipeline("x = print(42);")
441 
442 
443 if __name__ == "__main__":
444     unittest.main()
```

Listing 8.13: tests/test_pipeline.py — automated regression suite (75 tests)